A PROJECT REPORT ON

PREPMATE — AI INTERVIEW COACH

(COVER PAGE FORMAT FOR HARD BOUND)

A PROJECT REPORT ON

PREPMATE — AI INTERVIEW COACH

Submitted in partial fulfillment of the requirements for the award of the degree of

Master of Computer Applications (MCA)

Submitted By:

[Student Name]

[Roll Number]

Under the guidance of

[Guide Name]

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"PrepMate — AI Interview Coach"** is a bonafide record of work carried out by **[Student Name] ([Roll Number])** under my supervision and guidance in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA) during the academic year 2025-2026.

_____ _____

Signature of HOD Signature of Principal

STUDENT DECLARATION

I hereby declare that the project entitled “PrepMate — AI Interview Coach” submitted for the degree of Master of Computer Applications (MCA) is my original work and the project has not formed the basis for the award of any degree, diploma, fellowship, or similar other titles.

Place:

Date: _____

[Student Name]

CERTIFICATE FROM THE ORGANIZATION

[Include Company Letterhead Here]

This is to certify that [Student Name] has successfully completed their project titled 'PrepMate — AI Interview Coach' at [Company Name] from [Start Date] to [End Date].

_____

Authorized Signatory

[Company Name]

CERTIFICATE FROM THE PROJECT GUIDE

This is to certify that the project work entitled 'PrepMate — AI Interview Coach' is a bonafide work carried out by [Student Name] in partial fulfillment for the award of Master of Computer Applications (MCA) under my guidance and direction.

_____

[Guide Name]

[Designation]

[Department]

ACKNOWLEDGEMENT

I would like to express my deepest appreciation to all those who provided me the possibility to complete this project.

A special gratitude I give to my project guide, [Guide Name], whose contribution in stimulating suggestions and encouragement, helped me to coordinate my project effectively.

Furthermore, I would also like to acknowledge with much appreciation the crucial role of the staff and management of [Institution Name], who gave the permission to use all required equipment and the necessary materials to complete the task.

[Student Name]
[Roll Number]

ABSTRACT

The recruitment process in the tech industry has become increasingly competitive, demanding candidates to possess strong technical knowledge, problem-solving skills, and the ability to articulate their thoughts clearly. To address this, we developed PrepMate — an intelligent, full-stack AI interview preparation platform powered by Google Gemini.

PrepMate provides a comprehensive environment for candidates to practice tailored interview questions, receive AI-evaluated feedback, and track their progress over time. The platform features Resume-Based Interviews, wherein an uploaded PDF is parsed to generate questions targeting the user's exact tech stack. Spaced repetition algorithms ensure that failed questions are resurfaced until mastery is achieved. Furthermore, integrated voice transcription capabilities (via Groq API) allow users to simulate real spoken interviews.

The system is engineered using a modern architecture comprising a Spring Boot 4.1 backend (Java 21) and a React 19 frontend utilizing Vite. Data persistence is managed via PostgreSQL, while Pinecone vector databases are employed for semantic question deduplication. This project report details the system analysis, design, and implementation of PrepMate, demonstrating the effective integration of LLMs in educational and preparatory software.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

AI - Artificial Intelligence

API - Application Programming Interface

ATS - Applicant Tracking System

DDL - Data Definition Language

DFD - Data Flow Diagram

ER - Entity Relationship

JWT - JSON Web Token

LLM - Large Language Model

MCA - Master of Computer Applications

REST - Representational State Transfer

SPA - Single Page Application

UAT - User Acceptance Testing

UI - User Interface

CHAPTER 1

INTRODUCTION

1.1 Introduction

PrepMate is an intelligent, full-stack interview preparation platform specifically engineered and powered by Google Gemini. In today's highly competitive job market, software engineering candidates require more than just theoretical knowledge; they need robust, practical environments to simulate real-world technical and behavioral interviews. PrepMate is designed to bridge this gap by providing candidates with a comprehensive, centralized ecosystem where they can practice tailored interview questions, receive immediate AI-evaluated feedback, track their ongoing weaknesses, and master technical skills systematically. Unlike traditional platforms that offer static, generalized coding challenges, PrepMate introduces a dynamic, generative AI approach to mock interviews. It leverages a modern, highly scalable technology stack, combining a Spring Boot 4.1 backend utilizing Java 21 with a React 19 frontend built via Vite 8. This architecture not only guarantees high performance and low latency but also delivers a highly interactive, responsive educational tool capable of parsing resumes, evaluating nuanced human language, and processing audio transcriptions in real time.

By integrating state-of-the-art Large Language Models directly into its core workflow, PrepMate acts as a virtual, 24/7 technical interviewer. It eliminates the geographical and financial constraints typically associated with hiring expert human interviewers or subscribing to premium peer-to-peer mock interview platforms. The system is entirely stateless, secured via JSON Web Tokens (JWT), and uses PostgreSQL for robust relational data persistence, while employing Pinecone vector databases for advanced semantic deduplication of questions.

1.2 Background

Over the past decade, the technology landscape has evolved at an unprecedented pace, fundamentally altering the expectations placed upon software engineering candidates.

The barrier to entry for junior and mid-level roles has increased dramatically. Candidates are no longer merely expected to know basic syntax; they must demonstrate a broad and deep understanding of programming languages, complex frameworks, system design principles, and architectural scalability. Historically, candidates prepared for these rigorous hurdles by memorizing data structures and algorithms on platforms like LeetCode or HackerRank. While these tools are excellent for algorithmic problem-solving, they fall short in assessing a candidate's ability to communicate technical trade-offs, discuss past projects, or handle scenario-based architectural questions.

Traditional interview preparation methods have heavily relied on expensive human mock interviews or static question banks that quickly become outdated. Furthermore, the advent of Artificial Intelligence in education (EdTech) has historically been limited to simple keyword-matching chatbots or basic grammar correction tools. However, the recent breakthroughs in generative AI and Large Language Models (LLMs) have opened entirely new frontiers. Models like Google Gemini and OpenAI's GPT-4 possess the unprecedented ability to understand context, analyze complex code snippets, and generate human-like, nuanced conversational responses. PrepMate was conceptualized precisely at this intersection of advanced LLM capabilities and the glaring need for affordable, personalized interview preparation. It introduces a dynamic, generative AI-driven approach to mock interviews, ensuring that practice is adaptive, continually updated, and highly personalized.

1.3 Problem Statement

The primary problem addressing candidates today is the stark disconnect between how they practice and what they actually experience during a real technical interview. Candidates frequently struggle to find interview practice that accurately reflects their unique personal resumes, specialized experience levels, and targeted tech stacks. When a candidate applies for a specialized role—such as a Spring Boot Backend Developer or a React Frontend Engineer—they are often forced to practice generic questions that do not test their specific domain expertise.

Furthermore, existing platforms often provide generalized questions that lack real-time, constructive, and contextual feedback. If a candidate answers a conceptual question partially correct, static platforms either mark it completely wrong or offer no

feedback at all. Human interviewers provide subtle hints and grade on a spectrum, a nuance that traditional software fails to replicate. Candidates also lack a structured, automated mechanism to track their persistent weaknesses over time. Without an intelligent system to track which topics a candidate consistently fails, preparation becomes inefficient, leading to repeated mistakes and heightened anxiety during actual interviews.

1.4 Objectives

The primary objectives of the PrepMate project are meticulously aligned with solving the aforementioned problems through advanced software engineering and AI integration. The objectives are as follows:

1. Intelligent Question Generation: To develop a full-stack platform leveraging Google Gemini for generating dynamic interview questions that are strictly tailored by the candidate's chosen topic, experience level, and specific question style (Mixed, Definitions, Conceptual, or Scenario-Based).

2. Dynamic Resume Parsing: To implement Resume-Based Interviews using the Apache PDFBox library. This allows the system to extract text from uploaded PDF resumes dynamically, parse the candidate's actual projects and skills, and instruct the LLM to generate questions targeting their exact, unique tech stack.

3. Automated Evaluation Engine: To construct an AI Evaluation Engine capable of semantically analyzing user answers. The engine must score answers on a granular scale from 0 to 100, providing detailed, constructive feedback on what the candidate missed, alongside a perfect 'model answer' for immediate learning.

4. Spaced Repetition & Mastery Tracking: To build a robust spaced-repetition practice system using PostgreSQL for relational tracking, coupled with Pinecone vector embeddings. This ensures that questions scoring 70 or above are embedded and stored in Pinecone for semantic deduplication (preventing the AI from asking the same concept twice), while failed questions are continually resurfaced until mastery is achieved.

5. Real-Time Audio Transcription: To integrate the Groq Whisper API for transcribing real-time spoken voice answers, simulating the pressure, timing, and format of actual spoken interviews.

1.5 Scope of the Project

The scope of PrepMate encompasses the full software development lifecycle of a secure, cloud-ready web application. It includes the design and implementation of a RESTful API backend built on Spring Boot 4.1 and Java 21, and a responsive Single Page Application (SPA) frontend utilizing React 19 and Vite 8. The project focuses heavily on the intricate integration of third-party Artificial Intelligence APIs (Google Gemini, Groq Whisper) and specialized vector database storage (Pinecone).

The system scope includes comprehensive user management with JWT-based stateless authentication, ensuring secure and scalable access. It covers the creation of full interview session histories, allowing candidates to review past performances and per-question analytics. Additionally, the scope includes the implementation of a real-time 'Ask a Doubt' AI mentor module, which candidates can invoke during a session to receive immediate explanations for concepts they do not understand, further acting as an educational tool rather than just a testing platform.

1.6 Existing System

The existing ecosystem for technical interview preparation largely consists of platforms like LeetCode, HackerRank, and peer-to-peer networks like interviewing.io or Pramp. Systems like LeetCode rely entirely on static, hard-coded coding challenges that test algorithmic efficiency but fail completely at testing conceptual, architectural, or behavioral knowledge dynamically. They operate on strict unit tests, offering no leeway for semantic correctness or architectural discussions.

Conversely, peer-to-peer platforms rely on human engineers to conduct the interviews. While this provides realistic feedback, it introduces massive constraints regarding cost, scheduling, and consistency. A candidate may be paired with an overly harsh interviewer or someone unfamiliar with their specific tech stack. Neither of these existing paradigms comprehensively automates the generation of resume-specific technical questions while enforcing strict spaced-repetition mastery tracking.

They require manual effort from the candidate to identify their own weaknesses and manually seek out practice questions to patch those gaps.

1.7 Proposed System

The proposed system, PrepMate, completely revolutionizes and automates the mock interview lifecycle using generative AI. By integrating Google Gemini directly into the core Spring Boot backend architecture, the system acts as an infinitely scalable, highly knowledgeable virtual interviewer. When a candidate uploads their resume, PrepMate ingests the document, extracts the core competencies using Apache PDFBox, and prompts Gemini to formulate highly contextual, scenario-based questions that a real hiring manager would ask.

As the candidate responds—either by typing or by speaking into their microphone, which is transcribed in milliseconds via the Groq Whisper API—the system evaluates the response instantaneously. It grades the semantic accuracy of the answer, scores it, and provides immediate feedback. Furthermore, the proposed system introduces an intelligent architectural loop: questions that the candidate successfully masters (scoring 70 or above) are converted into vector embeddings and pushed to a Pinecone vector database. Before generating the next question, PrepMate queries Pinecone to ensure the new concept does not overlap semantically with already mastered concepts, thereby guaranteeing a highly efficient, non-repetitive learning curve.

1.8 Advantages of the Proposed System

The PrepMate architecture offers numerous distinct advantages over traditional preparation methods:

1. Unprecedented Personalization: Questions are formulated directly from the candidate's actual PDF resume, ensuring that a React developer is asked about React hooks, while a Java developer is grilled on JVM memory management.

2. Constructive Real-Time Feedback: The Gemini evaluation engine provides a granular 0-100 score along with a comprehensive 'model answer', allowing candidates to immediately recognize and rectify their knowledge gaps without waiting for human feedback.

3. Progressive Mastery & Efficiency: The integration of spaced repetition and Pinecone vector search ensures candidates focus strictly on their weak areas. The system actively prevents the repetition of concepts the candidate has already proven they understand, saving valuable preparation time.

4. Realistic Interview Simulation: Support for voice answers via the Groq Whisper API forces candidates to practice verbalizing complex technical concepts, simulating the exact pressure and format of real spoken interviews.

5. Security, Scalability, and Availability: Built on a stateless JWT architecture using Spring Boot 4.1 and React 19, the platform allows for seamless horizontal scaling. Candidates have 24/7 access to an expert AI interviewer, completely eliminating scheduling conflicts and hourly costs associated with human mentors.

CHAPTER 2

LITERATURE SURVEY

2.1 Review of Related Work

The application of Artificial Intelligence in education (EdTech) and professional preparation has seen exponential growth over the last decade. Historically, candidates preparing for technical interviews have relied on static coding platforms, such as LeetCode and HackerRank, to practice algorithmic problems. While highly effective for data structures and algorithms, these platforms primarily focus on compiler-based evaluation, grading code against predefined test cases. They lack the semantic understanding necessary to evaluate architectural choices, system design, or behavioral responses.

To bridge this gap, peer-to-peer and expert-led mock interview platforms like interviewing.io and Pramp emerged. These platforms simulate the authentic interview environment by pairing candidates with human engineers. However, literature highlights several critical limitations of human-led mock interviews, including high costs, scheduling constraints, subjective biases, and the inability to guarantee an interviewer's expertise in the candidate's exact tech stack. Recent academic discourse on Natural Language Processing (NLP) has suggested that intelligent conversational agents can mitigate these issues by acting as scalable, objective evaluators.

Early attempts at NLP-driven mock interviewers utilized simplistic keyword-matching algorithms and predefined decision trees. These systems were notoriously brittle; if a candidate explained a concept using a synonym or a slightly different architectural approach, the system would fail to recognize the correct answer. The critical turning point in the literature is the introduction of Large Language Models (LLMs) featuring Transformer architectures, which demonstrate profound capabilities in contextual understanding and semantic analysis.

2.2 Existing Technologies

Most current software solutions employ rudimentary static algorithms to serve practice questions. In these systems, question selection is often randomized or based on basic tags (e.g., 'Easy', 'Medium', 'Hard'). They do not dynamically adapt to a candidate's specific background or track nuanced progress over time.

With the advent of Large Language Models (LLMs) such as OpenAI's GPT-4 and Google's Gemini, the technological capability to comprehend nuanced, context-heavy technical dialogue has drastically improved. Gemini, specifically, offers robust multi-modal capabilities and high-speed token generation, making it uniquely suited for real-time educational feedback. Furthermore, the integration of Voice-to-Text APIs, such as the Groq Whisper model, represents a major technological leap. Whisper enables highly accurate, low-latency audio transcription, allowing software to evaluate spoken technical jargon seamlessly.

Another critical existing technology is the Vector Database, heavily discussed in recent AI literature. Traditional relational databases (like PostgreSQL or MySQL) are excellent for structured data but inefficient at searching for semantic similarities. Technologies like Pinecone allow text strings (such as interview questions) to be converted into high-dimensional numerical vectors (embeddings). This allows a system to mathematically calculate the 'distance' or similarity between two concepts. In the context of PrepMate, Pinecone is leveraged to ensure that once a candidate masters a specific concept, the system will not generate semantically identical questions, enforcing true spaced repetition.

2.3 Comparison of Previous Systems

PrepMate distinguishes itself fundamentally from existing systems by aggregating Google Gemini for real-time generative evaluation, Pinecone for vector-based spaced repetition, and Apache PDFBox for direct resume integration. Unlike generic LLM chat interfaces (like ChatGPT), PrepMate strictly structures the interview flow, enforces a 0-100 scoring rubric, and permanently tracks mastery.

Table 2.1 provides a detailed comparison between traditional platforms, human-led

mock interviews, and the proposed PrepMate system.

Table 2.1: Comparison of Interview Preparation Platforms

Feature Dimension	Static Platforms (LeetCode)	Human Mock Interviews (Pramp)	Proposed System (PrepMate)
Personalization	Low (Generic question banks)	Medium (Depends on peer expertise)	High (Generates questions directly from PDF resume)
Evaluation Type	Unit test compiler execution	Subjective human feedback	AI semantic scoring (0-100) + Model Answer
Cost & Availability	Low cost / 24/7 Availability	High cost / Scheduling Required	Low cost / 24/7 Availability
Spaced Repetition	Manual tracking required	None	Automated via Pinecone Vector Database
Voice Support	No	Yes (Live call)	Yes (Groq Whisper audio transcription)

As evident from Table 2.1, PrepMate successfully merges the 24/7 availability and low cost of static platforms with the personalized, nuanced feedback of human mock interviews, augmented by vector-driven progress tracking.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Requirement Analysis

Requirement analysis is the most critical phase in the Software Development Life Cycle (SDLC) for PrepMate. It involved identifying the primary actors, defining the system boundaries, and establishing the core capabilities required to facilitate a robust, generative mock interview environment. The primary actors in PrepMate include the Candidate (End-User) and the AI Services (Google Gemini, Groq, Pinecone). The analysis phase determined that the system must strictly handle multi-modal inputs—specifically, standard text inputs and audio vocalizations—and process unstructured data such as PDF resumes. The requirement gathering process emphasized a need for real-time responsiveness; since the system acts as a live interviewer, the latency between a user's answer and the AI's subsequent question must mimic human conversation as closely as possible.

3.2 Functional Requirements

Functional requirements define the exact technical capabilities and behaviors the system must exhibit. For PrepMate, these were mapped directly to the interaction between the React frontend and the Spring Boot API:

• FR-01 User Authentication: The system shall allow users to register and login using stateless JSON Web Token (JWT) based authentication, ensuring session security without server-side state.

• FR-02 Resume Parsing: The system shall securely accept PDF uploads, utilize Apache PDFBox to extract the unstructured text, and parse out core competencies to guide the LLM.

• FR-03 AI Question Generation: The system shall formulate highly contextual, scenario-based, and conceptual interview questions utilizing the Google Gemini API, tailoring the difficulty to the candidate's historical performance.

• FR-04 Voice Transcription: The system shall capture microphone audio via the browser, transmit the binary data to the backend, and transcribe the spoken answers to text using the Groq Whisper API.

• FR-05 AI Evaluation & Scoring: The system shall evaluate the transcribed or typed answers, score them on a 0-100 rubric, and return a comprehensive 'model answer' for candidate review.

• FR-06 Spaced Repetition Mastery: The system shall store mathematical vector embeddings of mastered concepts (score >= 70) in Pinecone, actively querying this database to prevent asking redundant questions.

Table 3.1: Functional Requirements Traceability Matrix

Req ID	Feature Description	Target Subsystem
FR-01	JWT Authentication & Authorization	Spring Security / AuthController
FR-02	PDF Text Extraction	Apache PDFBox Service
FR-03	LLM Question Generation	Gemini API Service
FR-04	Audio to Text Transcription	Groq Whisper API
FR-05	0-100 Answer Evaluation	Gemini API Service
FR-06	Semantic Deduplication	Pinecone Vector DB

3.3 Non-Functional Requirements

Non-functional requirements dictate system attributes such as performance, usability, and security. They form the architectural constraints of PrepMate:

• NFR-01 Performance (Latency): Because LLM generation is inherently slow, the API responses for question generation and evaluation must be heavily optimized, targeting completion within < 3 seconds to maintain conversational flow.

• NFR-02 Security: All application endpoints, excluding the public registration and login routes, must be strictly secured. Passwords must be hashed using BCrypt before persistence in PostgreSQL.

• NFR-03 Scalability: By adopting a completely stateless architecture with JWT, the Spring Boot application must be capable of horizontal scaling behind a load balancer to accommodate concurrent mock interviews without session replication issues.

• NFR-04 Maintainability: The codebase must adhere to strict Separation of Concerns (SoC), isolating Controller, Service, and Repository layers in Java, and utilizing component-driven architecture in React 19.

3.4 Feasibility Study

A comprehensive feasibility study was conducted prior to development to ensure that PrepMate could be successfully engineered given resource, time, and technological constraints. The study evaluated three primary domains: Technical, Economic, and Operational feasibility.

3.4.1 Technical Feasibility

The technical feasibility assessment evaluated whether the proposed technology stack (Java 21, Spring Boot 4.1, React 19) was robust enough to handle the complex AI integrations. The study concluded that utilizing Spring WebFlux or standard synchronous REST with advanced multithreading would comfortably handle the API calls to Gemini and Groq. Furthermore, integrating Pinecone for vector embeddings abstracts away the immense complexity of building a local semantic search engine, rendering the vector-based spaced repetition highly technically feasible. The open-source nature of Apache PDFBox guarantees that resume parsing does not require expensive proprietary licenses.

3.4.2 Economic Feasibility

The economic feasibility study analyzed the cost of operating the platform. Unlike traditional startups that require massive upfront capital for server racks, PrepMate utilizes a highly economical cloud-native stack. PostgreSQL is open-source and free. The Google Gemini API offers a generous free tier for development, and the Groq Whisper API provides hyper-fast transcription at a fraction of the cost of legacy providers. Because the system replaces human interviewers—who command high hourly rates—the platform is incredibly economically feasible and highly scalable with minimal marginal cost per user.

3.4.3 Operational Feasibility

Operational feasibility evaluates how well the proposed system solves the candidate's problems and how easily it can be adopted. Candidates are already accustomed to web-based code editors and chat interfaces. By providing a clean, intuitive SPA built with React 19 and Vite, the learning curve is practically non-existent. Candidates simply upload a resume and begin talking or typing. The automated tracking of weaknesses operates entirely in the background via Pinecone, requiring zero operational overhead from the user. Thus, the system is highly operationally feasible.

Table 3.2: Feasibility Assessment Matrix

Feasibility Type	Assessment Criteria	Risk Level	Mitigation Strategy
Technical	Integration of 3 distinct external APIs (Gemini, Groq, Pinecone)	Medium	Use distinct Spring Services for isolation and error handling
Economic	API usage limits and costs during peak scale	Low	Implement rate-limiting and utilize Gemini free-tier quotas
Operational	User adoption and UI complexity	Low	Leverage React 19 for a highly responsive, modern, and familiar chat UI

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

System architecture defines the foundational structural blueprint and high-level design of the application. It meticulously details how discrete software modules, backend databases, and external artificial intelligence APIs communicate to fulfill the functional and non-functional requirements. PrepMate is engineered using a highly decoupled, modern Client-Server architectural pattern. By enforcing strict Separation of Concerns (SoC), the system ensures that changes in the user interface do not mandate complex refactoring in the backend processing logic.

The system is structurally divided into three primary, robust tiers: the Presentation Layer (Frontend), the Business Logic Layer (Backend REST API), and the Data Access Layer (Persistence & Semantic Embeddings).

The Presentation Layer acts as the primary user interface. It is developed as a highly responsive Single Page Application (SPA) using React 19 and bundled via Vite 8. This layer is responsible for handling all user interactions, orchestrating browser-native audio recording for voice-based interviews, and dynamically rendering AI feedback without full-page reloads. The use of functional React components ensures modularity and reusability across the application.

The Business Logic Layer represents the core intelligence and orchestration engine of PrepMate. It is constructed as a stateless Spring Boot 4.1 application running on the Java 21 runtime. This layer enforces JWT-based authentication, secures endpoints, and acts as the central middleware. It orchestrates highly complex external HTTP calls to the Google Gemini LLM for generative answer evaluation. Additionally, it implements advanced file processing by utilizing Apache PDFBox to parse text from uploaded resumes, effectively converting unstructured candidate data into structured LLM prompt constraints.

Finally, the Data Access Layer implements an advanced dual-database strategy to handle distinct types of data persistence. A PostgreSQL relational database is utilized to maintain highly structured transactional data, including user credentials, detailed session histories, and lists of mastered questions. Conversely, a Pinecone Vector Database is integrated to handle high-dimensional mathematical data. The Pinecone database stores the semantic vector embeddings of technical concepts, enabling the system's spaced-repetition algorithm to mathematically calculate the conceptual similarity between past questions and newly generated questions.

Figure 4.1 illustrates the high-level architecture component mapping of PrepMate:

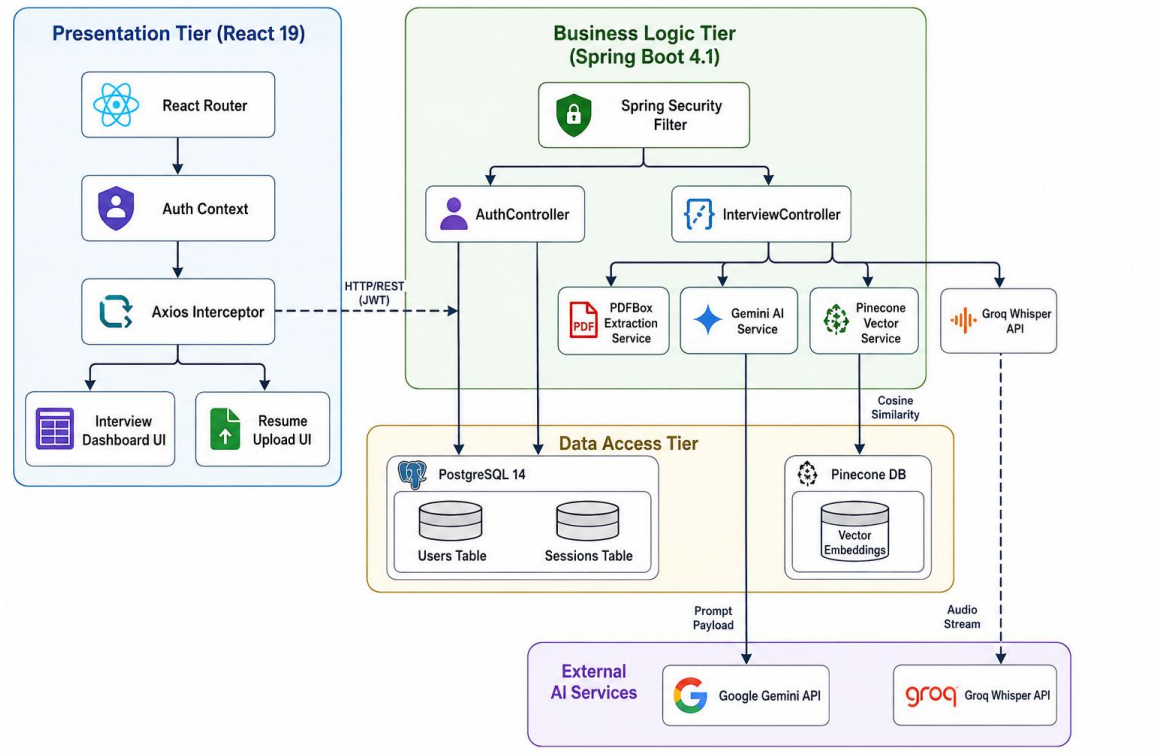

Figure 4.1: System Architecture Diagram

4.2 UML Diagrams

The Unified Modeling Language (UML) diagrams provide a standardized, visual representation of the system's structural layout and behavioral execution. These models are crucial for understanding the complex interplay between object-oriented classes, database entities, and asynchronous API flows within the PrepMate ecosystem.

By mapping the system visually, developers and stakeholders can ensure that the theoretical design accurately reflects the intended real-world operational flows before intensive coding begins.

4.2.1 Use Case Diagram

The Use Case Diagram defines the critical interactions between the primary actors and the overarching boundaries of the system. In the context of PrepMate, the primary human actor is the Candidate. The Candidate initiates all primary interactions with the system, including managing their authenticated profile, initiating new dynamic mock interviews, submitting both audio and text-based answers, and reviewing their historical performance metrics.

The system boundaries encapsulate highly complex computational use cases such as 'Generate Contextual Interview', 'Evaluate Answer Semantics', and 'Track Spaced Repetition Mastery'.

Furthermore, a unique aspect of this architectural Use Case Diagram is the explicit inclusion and mapping of secondary, non-human system actors. These actors include the Google Gemini API, the Groq Whisper Audio Transcription API, and the Pinecone Vector Database. The diagram effectively illustrates how the core Spring Boot system acts as a central operational orchestrator, actively delegating heavy computational tasks—such as Natural Language Processing and Vector Cosine Similarity calculations—to these highly specialized external microservices.

From a business and functional perspective, this Use Case Diagram explicitly bounds the scope of the Minimum Viable Product (MVP). By strictly defining what the Candidate can and cannot do, it prevents feature creep. For instance, the diagram purposefully omits administrative moderation use cases, focusing entirely on the high-

value generative workflows that differentiate PrepMate in the educational technology market.

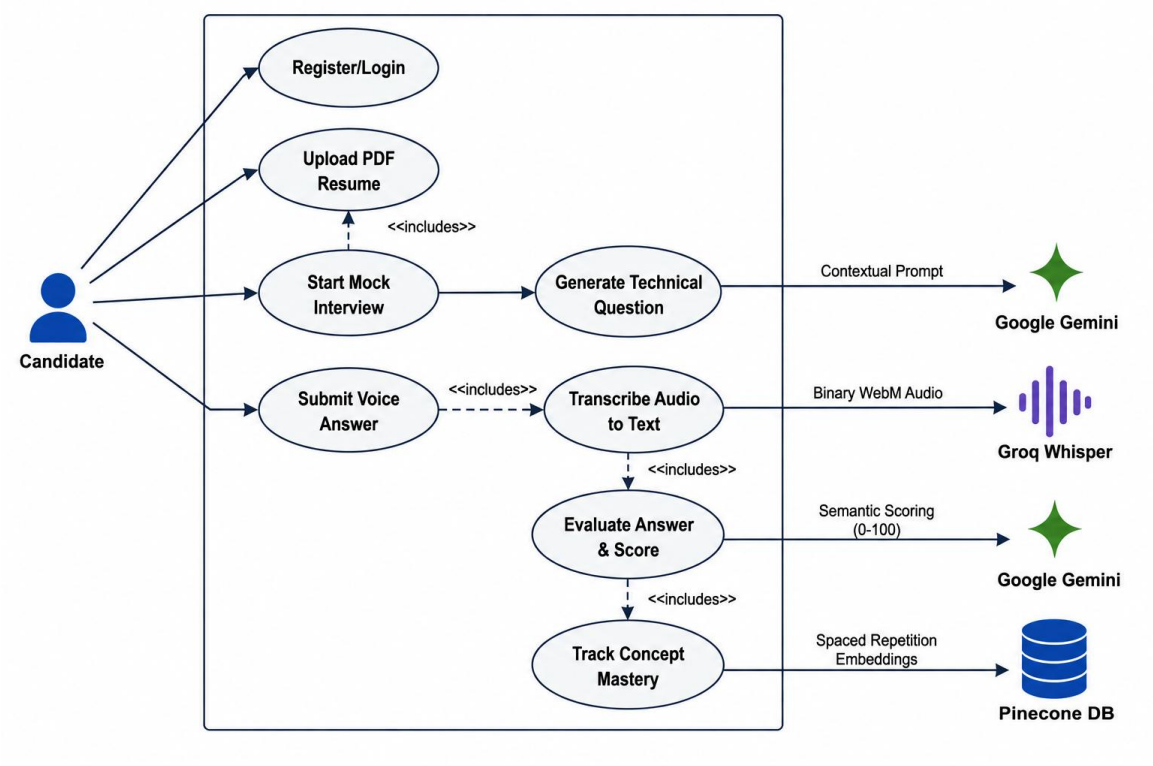

Figure 4.2: Use Case Diagram

4.2.2 Class Diagram

The Class Diagram illustrates the static, object-oriented structure of the backend Spring Boot application. It details the precise structural relationships, inheritance hierarchies, and dependencies between the Java Controllers, Service Interfaces, and JPA Repositories.

The diagram specifically highlights the core 'Interview' architecture, showcasing the 'InterviewController' which exposes the core RESTful endpoints to the React frontend. This controller is structurally mapped via Dependency Injection to backend computational services, notably the AI services and Vector databases.

From a persistence standpoint, the Class Diagram demonstrates the strict Entity relationships. It visualizes how a singular 'User' entity possesses a one-to-many relationship with multiple 'InterviewSession' entities, which in turn aggregate multiple 'QuestionEvaluationEntity' objects. This granular structural mapping ensures strict adherence to SOLID object-oriented design principles by clearly demarcating data access objects from the business logic orchestrators.

Technically, this diagram serves as a blueprint for the backend engineering team. By utilizing interfaces for the external API integrations (such as an LLMProvider interface, implicitly represented by the Service layer), the Class Diagram dictates a loosely coupled architecture. This means that if the business eventually decides to migrate away from Google Gemini to OpenAI's GPT models, the underlying Class architecture requires minimal refactoring, proving the robustness of the chosen design patterns.

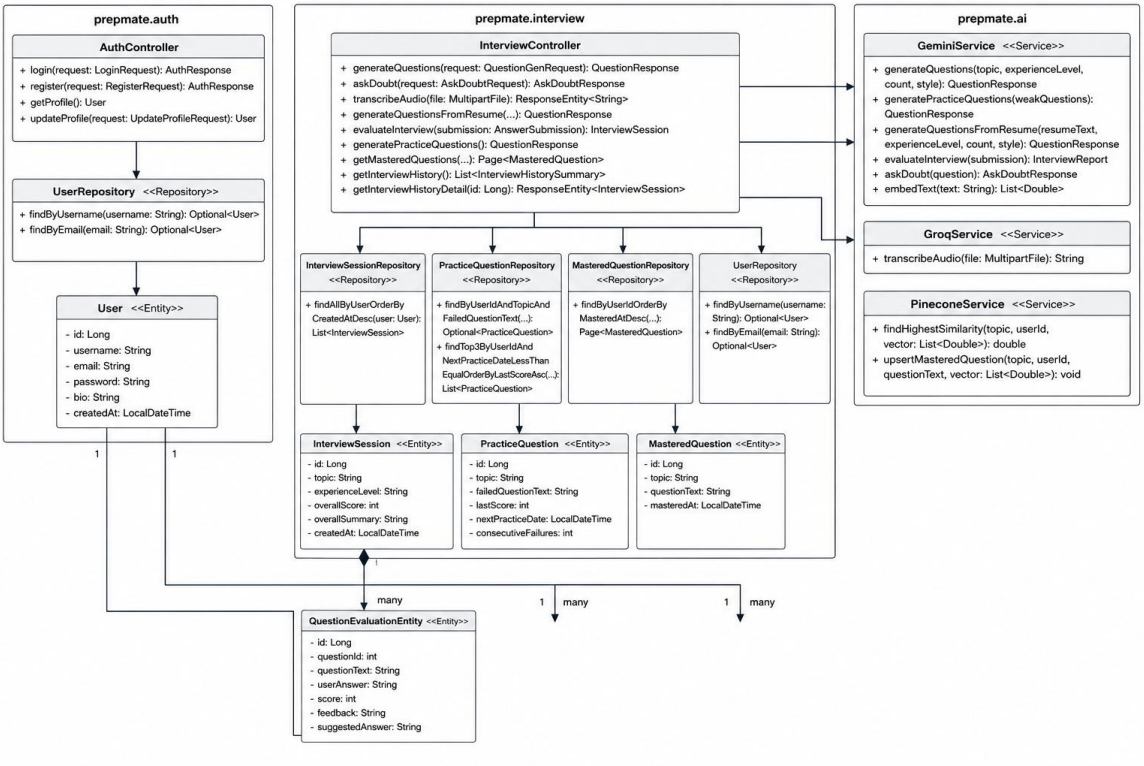

Figure 4.3: Class Diagram

4.2.3 Sequence Diagram (Answer Evaluation)

Sequence Diagrams trace the chronological execution path and strict message passing between discrete objects and network boundaries over time. Figure 4.4 details the highly complex, multi-tiered asynchronous flow of the Answer Evaluation process.

The sequence initiates when a candidate submits a spoken audio answer. The React frontend captures this via the MediaRecorder API and transmits a binary WebM file payload across the network to the Spring Boot backend. The backend, acting as an intermediary, concurrently streams this payload to the Groq API endpoint for rapid, low-latency audio transcription.

Upon receiving the successfully transcribed string of text, the Spring Boot application constructs a highly specialized, context-aware prompt. This payload contains the candidate's transcribed answer, the original generated question, and the current state of the interview. This payload is dispatched to the Google Gemini API. Once Gemini processes the semantic validity of the answer and returns a structured evaluation containing a numerical score and constructive feedback, the system maps this payload to a JPA entity, persists the state transaction in PostgreSQL, and finally returns a formalized JSON response back to the React frontend to update the User Interface dynamically. This Sequence Diagram effectively highlights the critical, time-sensitive API handoffs required to maintain the illusion of a real-time, human-like interview simulation.

A critical takeaway from this Sequence Flow is the sheer volume of asynchronous network hops required for a single interaction. Because audio processing, LLM generation, and vector database lookups are fundamentally I/O bound operations, the sequence diagram dictates that the backend must utilize non-blocking, asynchronous threading models (such as Java Virtual Threads) to prevent thread starvation and ensure the system can scale horizontally under high concurrent candidate loads.

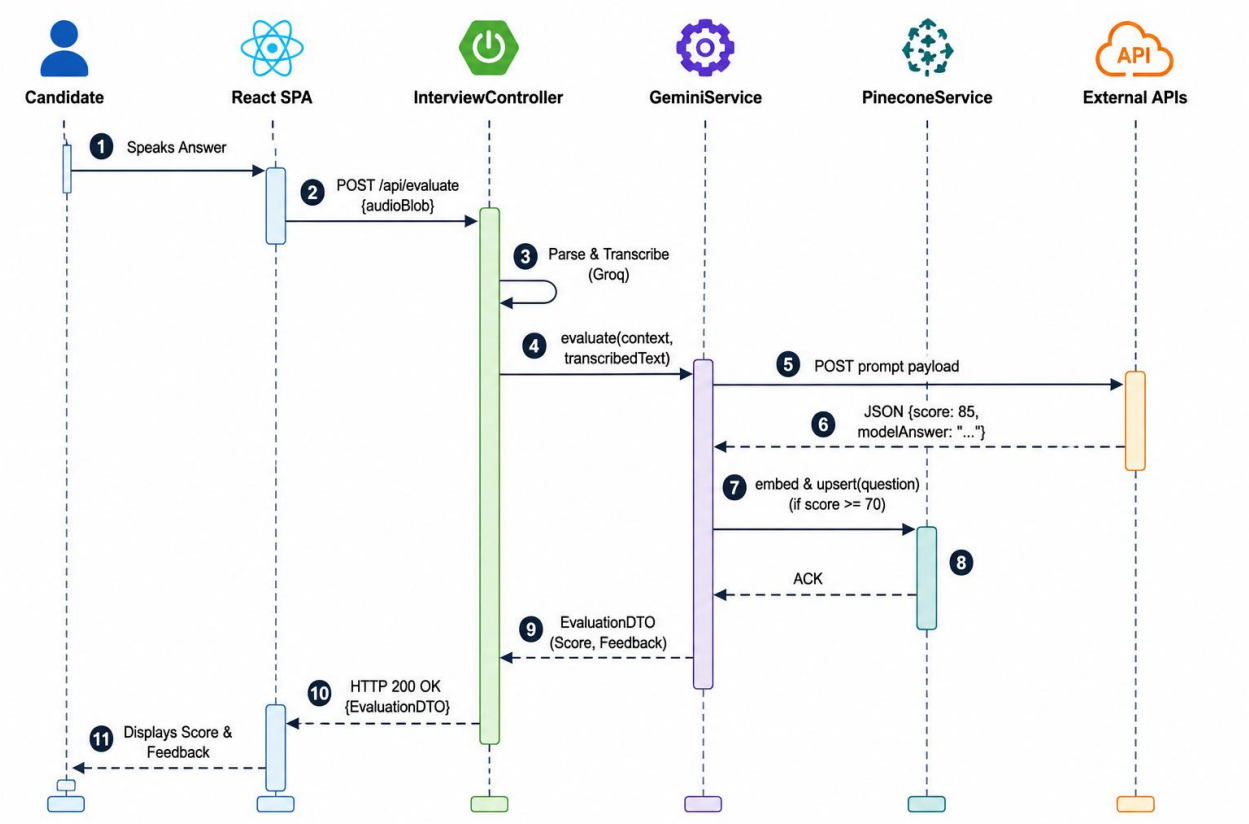

Figure 4.4: Sequence Flow Diagram

4.2.4 Activity Diagram 1: Overall Flow

Activity Diagrams map the algorithmic decision-making branches, parallel processing paths, and control flows within the system architecture. The Overall Flow diagram illustrates the macro-level lifecycle of an interview session from initialization to termination.

The control flow captures the initial critical decision node where the system evaluates if the requested interview is strictly resume-based or generic. If resume-based, the flow branches into specialized PDF parsing logic utilizing Apache PDFBox; otherwise, it proceeds to standard generative prompt flows. It further illustrates the continuous operational loop: dynamically generating a contextual question, pausing execution to await candidate input, dispatching the response for LLM evaluation, and subsequently updating the relational weakness queue based on whether the candidate's score exceeded the mastery threshold. This specific diagram provides developers with a holistic, top-down view of the system's operational lifecycle logic.

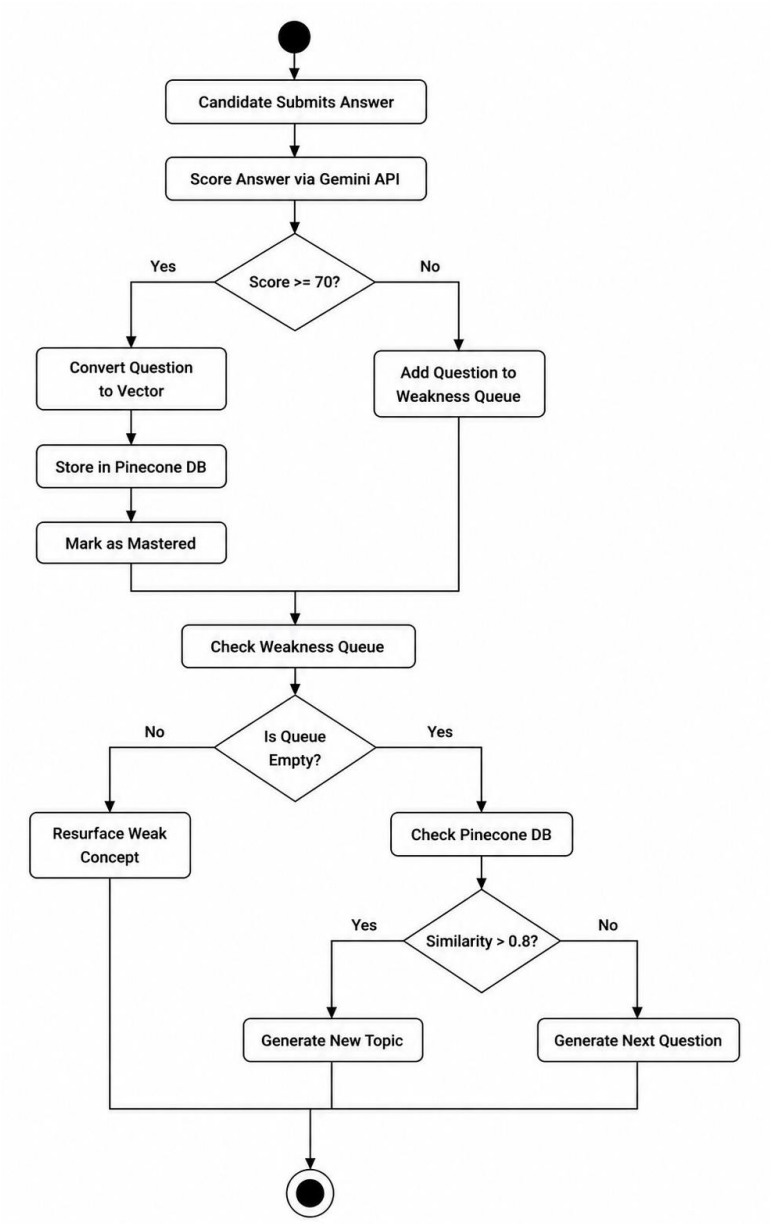

Figure 4.5: Activity Diagram 1 (Overall Flow)

4.2.5 Activity Diagram 2: Core AI Interview & Evaluation

This specialized Activity Diagram dives significantly deeper into the micro-level control flow of the core Artificial Intelligence evaluation mechanism. It meticulously outlines the step-by-step conditional logic from the exact moment the candidate speaks an answer into their microphone.

The diagram visualizes critical conditional routing and error handling: if the Groq transcription fails or returns an empty string, an error state is propagated back to the user; if successful, the transcribed text is actively evaluated by the LLM. Crucially, this control flow maps the complex branching logic based on the specific interview context—demonstrating how the system intentionally bypasses vector mastery tracking if the interview is strictly resume-based, or actively triggers the spaced repetition vectorization logic if it is a standard, conceptual technical interview.

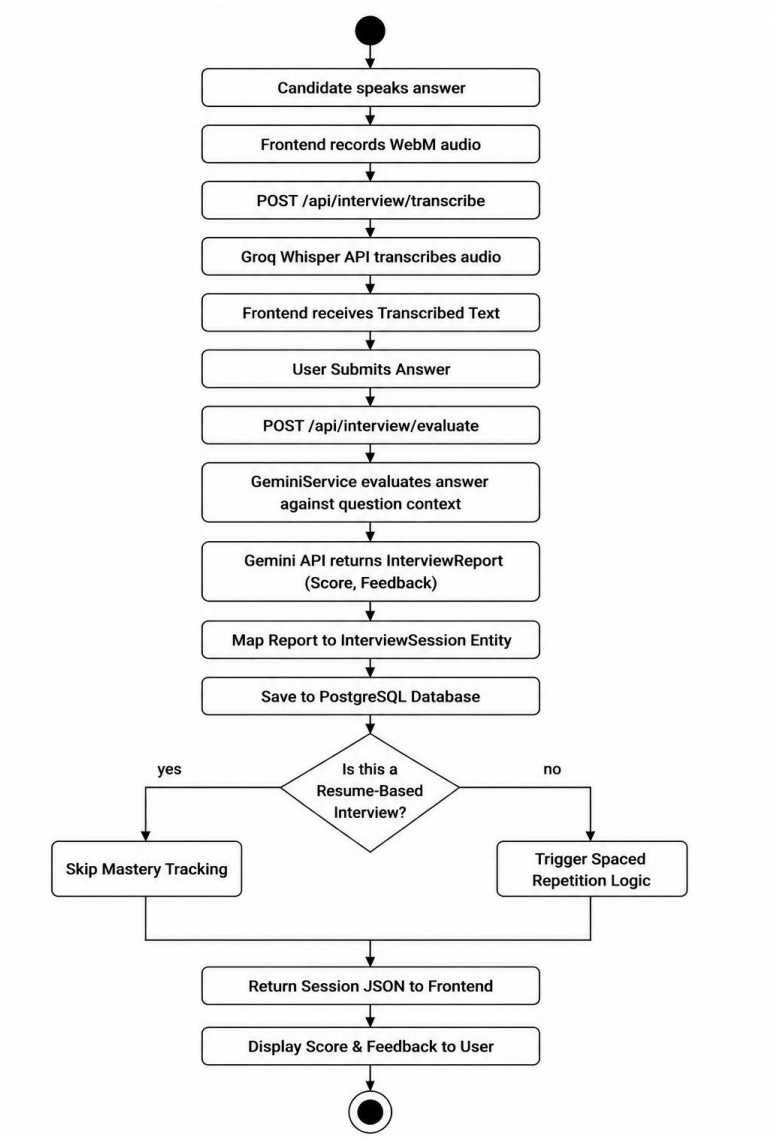

Figure 4.6: Activity Diagram 2 (Core Evaluation)

4.2.6 Activity Diagram 3: Spaced Repetition & Generation

The Spaced Repetition Activity Diagram maps the highly advanced, mathematically driven algorithmic loop responsible for dynamically generating unique, non-repetitive interview questions. This flow is the cornerstone of PrepMate's adaptive learning engine.

When the system needs to formulate a new question, it prompts Google Gemini to generate a batch of potential, contextually relevant candidates. For each candidate question generated, the system utilizes a lightweight embedding model to convert the text string into a high-dimensional vector array. It then queries the Pinecone Vector Database using this embedding.

The diagram features a highly critical decision node where the system calculates the Cosine Similarity mathematically against previously mastered concepts stored in the database. If the returned similarity index is above the strict 0.85 threshold, the generated question is algorithmically rejected as being too redundant or conceptually identical to something the candidate already knows. If the similarity is below the threshold, the question is accepted. This complex control flow ensures the candidate is constantly challenged with novel technical concepts, entirely eliminating the repetition of trivial knowledge.

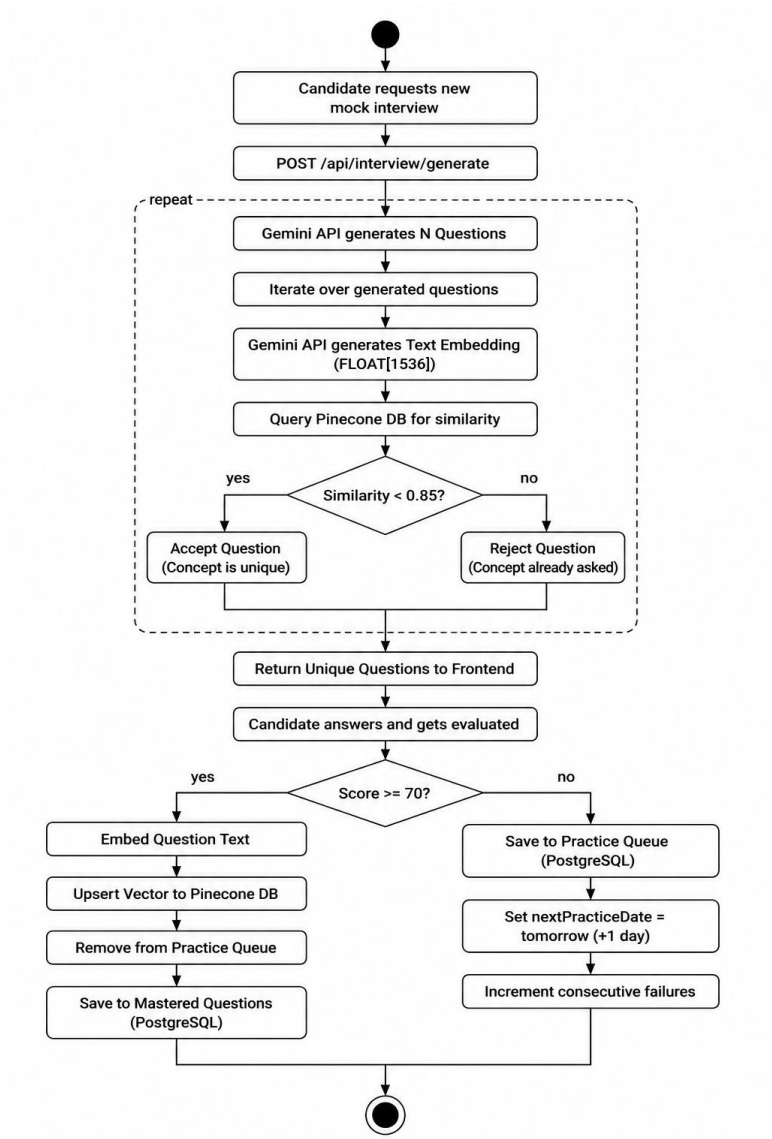

Figure 4.7: Activity Diagram 3 (Spaced Repetition)

4.3 Database Design (Entity-Relationship Diagram)

The Database Design section outlines the logical structure and referential constraints of the application's persistent data layer. To guarantee data integrity, the relational schema in PostgreSQL is heavily normalized to the Third Normal Form (3NF). This normalization strategy effectively eliminates data redundancy and ensures strict referential integrity across all transactional records.

The Entity-Relationship (ER) Diagram maps the core domain entities, which include 'users', 'interview_sessions', 'practice_questions', and 'mastered_questions'. Primary keys, generated automatically as BIGINT IDs, and explicit foreign key constraints link the vast amounts of historical session data directly back to the authenticated user owning the data.

Additionally, the ER diagram intricately illustrates the associative tables utilized to track session-specific strengths, weaknesses, and LLM-generated improvement areas. By utilizing a robust, ACID-compliant database like PostgreSQL, the system completely guarantees transactional integrity for all critical user records, while purposefully offloading all unstructured semantic, non-relational vector data to the Pinecone NoSQL database for rapid, specialized mathematical querying.

The architectural decision to heavily normalize the SQL tables while keeping the vector data isolated in Pinecone is a direct response to the hybrid nature of the application. The ER diagram proves that the system maintains a relational source of truth for billing, analytics, and session history, while remaining agile enough to perform unstructured semantic lookups on the fly.

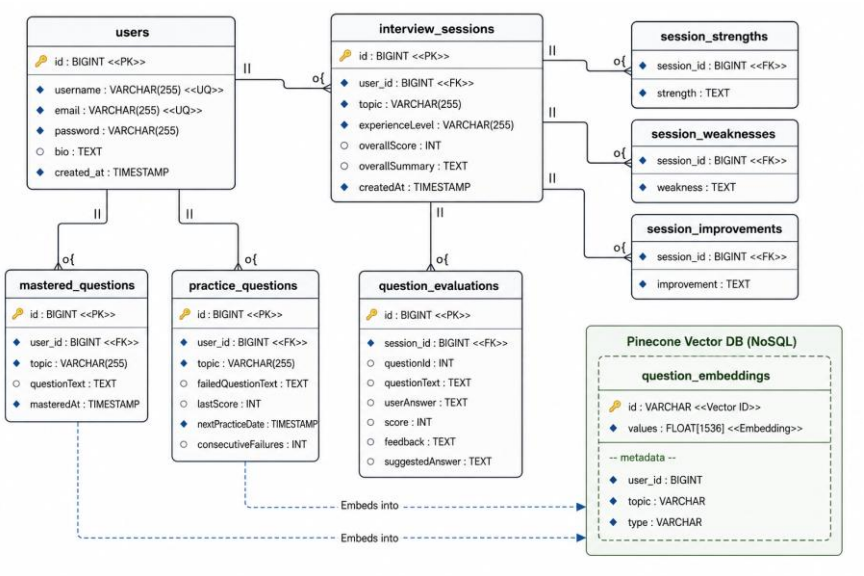

Figure 4.8: Entity-Relationship (ER) Diagram

4.4 Data Flow Diagrams

Data Flow Diagrams (DFDs) provide a graphical representation of the "flow" of data through an information system, modeling its process aspects. Unlike control flow diagrams, DFDs do not represent the sequence of operations or decision logic; instead, they focus strictly on what data goes in, what data goes out, where it is stored, and which external entities are involved. In the context of PrepMate, these diagrams are essential for identifying the precise data contracts and API payloads that must be transmitted securely across network boundaries.

The Level 0 Context Diagram (Figure 4.9) represents the entire PrepMate software ecosystem as a single, unified macro-process. This high-level view is primarily designed for non-technical stakeholders to quickly grasp the system boundaries. It explicitly maps the flow of data originating from the external Candidate actor—specifically, their uploaded PDF resumes and spoken WebM audio answers—into the core system. It further illustrates how the core system distributes this data to specialized external AI agents (Google Gemini and Groq) before ultimately returning structured AI feedback and numerical scores back to the Candidate.

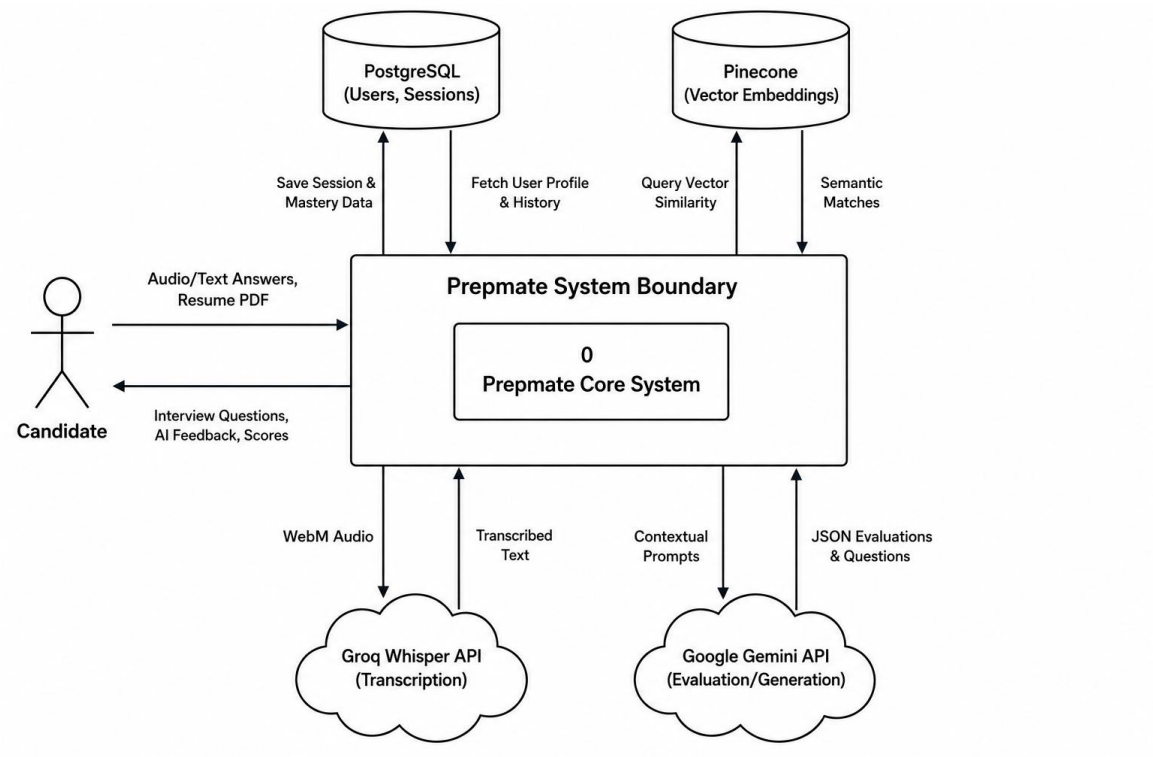

Figure 4.9: Level 0 Context DFD

The Level 1 Data Flow Diagram (Figure 4.10) deconstructs the single Level 0 process into its constituent sub-processes, exposing the internal data routing architecture. It breaks the system down into five major processing nodes: (1.0) User Authentication, (2.0) Interview Setup & Parsing, (3.0) Audio Transcription, (4.0) AI Evaluation, and (5.0) Spaced Repetition Generation.

This deeper architectural view explicitly maps how data stores are utilized across different stages of the application lifecycle. For example, it illustrates how JWT credentials flow through the Authentication process to validate against the PostgreSQL 'Users' table. More critically, it visualizes the complex data pipeline where transcribed text flows into the AI Evaluation process, generating a JSON Score that is subsequently persisted in PostgreSQL, while simultaneously feeding the Spaced Repetition process to query mathematical similarity vectors from the Pinecone Database. By distinctly mapping these data pipelines, developers can guarantee that sensitive candidate data is properly routed, encrypted, and persisted across the distributed microservice architecture.

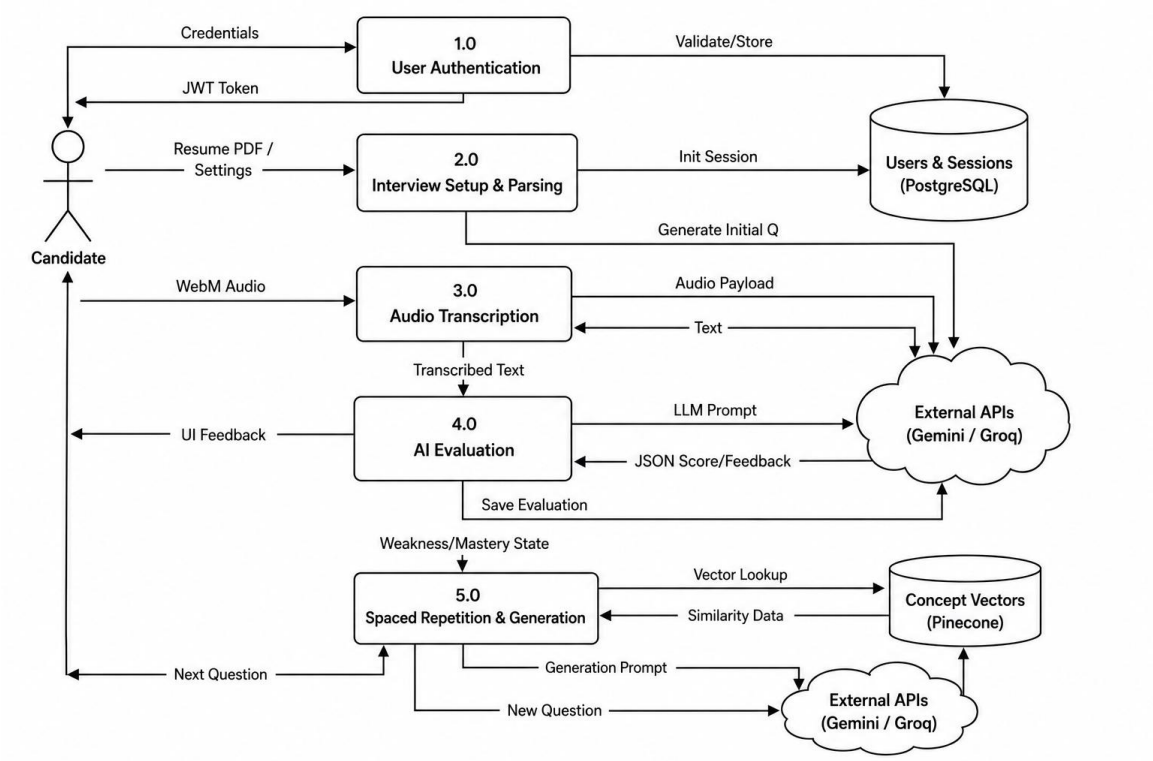

Figure 4.10: Level 1 Data Flow Diagram

CHAPTER 5

SYSTEM IMPLEMENTATION

The implementation phase of the software development life cycle (SDLC) translates the theoretical system architecture into executable source code. For PrepMate, this required establishing a highly robust, full-stack development environment utilizing modern enterprise technologies.

5.1 Hardware Requirements

To ensure a smooth local development and compilation experience—especially when running concurrent Docker containers alongside heavy Java Virtual Machines and Node.js instances—the following minimum hardware specifications were mandated for the PrepMate project:

Component	Minimum Specification	Justification
Processor (CPU)	Multi-core Intel Core i5 / AMD Ryzen 5 or Apple Silicon (M1+)	Required to support high concurrent thread context switching for Virtual Threads.
Memory (RAM)	16 GB DDR4/DDR5	The Spring Boot backend context, combined with Vite HMR and Dockerized PostgreSQL, requires substantial volatile memory.
Storage	50 GB Solid State Drive (SSD)	Fast I/O is critical for reducing Maven compilation times and accelerating Docker image building.
Network	High-bandwidth, stable	Relies heavily on

	broadband	continuous real-time communication with Google Gemini, Groq Whisper, and Pinecone.

5.2 Software Requirements

PrepMate relies on a meticulously curated stack of modern, enterprise-grade software technologies designed for extreme scalability.

Technology	Version / Framework	Purpose & Justification
Backend Runtime	Java 21 (JDK)	Native support for Virtual Threads (Project Loom) to prevent thread exhaustion during high-concurrency LLM calls.
Backend Framework	Spring Boot 3.x	Provides the application context, Spring Web for REST APIs, and Spring Data JPA for ORM.
Frontend Runtime	Node.js (v20+)	Executes the Vite build pipeline and handles local development server routing.
Frontend Framework	React 19 & Vite 8	React provides functional UI components; Vite ensures blazing-fast Hot Module Replacement (HMR).
Styling	Tailwind CSS 4	Atomic, utility-first CSS framework for rapid UI styling without massive external stylesheets.
Relational Database	PostgreSQL 14+	ACID-compliant data

		persistence for users, billing, and interview history.
Vector Database	Pinecone	Serverless NoSQL database designed specifically for mathematical Cosine Similarity querying on floating-point arrays.

5.3 Development Environment

The development environment for PrepMate was designed to bridge the gap between local programming and production cloud deployment. For the backend, IntelliJ IDEA was the primary Integrated Development Environment (IDE), providing deep integrations with the Maven build lifecycle and Spring Boot application contexts. Maven was utilized strictly for dependency management (fetching JARs from the Maven Central Repository) and orchestrating the build lifecycle.

On the frontend, Visual Studio Code (VSCode) was utilized alongside the Vite build tool. Vite was selected specifically for its instantaneous Hot Module Replacement (HMR) capabilities, allowing developers to see React UI updates in milliseconds without full page reloads. Finally, Docker was utilized extensively. By containerizing the PostgreSQL database in a Docker image, the team ensured that the local development database environment perfectly mirrored the Neon Cloud PostgreSQL environment used in production.

5.4 Modules

The PrepMate backend is divided into separate, easy-to-understand functional blocks called modules. By splitting the application into these modules, the software becomes much easier to maintain, test, and upgrade over time. Each module has a very specific, single responsibility.

5.4.1 Security & Authentication Module

This module is the digital bouncer for the PrepMate platform. Its sole responsibility is to ensure that only registered and verified users can access the system. When a user logs in, this module securely checks their email and password against the database. If they match, it generates a secure, encrypted 'digital ID badge' (known as a JSON Web Token). The user's browser holds onto this badge and shows it to the server every time they try to load a new page or save their interview results, ensuring absolute data privacy and security for all candidates.

From a business perspective, this module is critical because it protects user data from unauthorized access, ensuring that candidates feel safe uploading their personal resumes and recording their voices.

5.4.2 AI Integration Module

This module acts as the communication bridge between PrepMate and the outside world of Artificial Intelligence. Instead of forcing the core PrepMate application to understand the complex, low-level network details of how to talk to Google Gemini or Groq, this module handles all of that heavy lifting. It safely packages up the user's answers, sends them over the internet to the AI providers, and translates the AI's responses back into a format that PrepMate can easily read.

By isolating all AI communication into this single module, the engineering team can easily swap out the AI providers in the future. For example, if a better AI than Google Gemini is released next year, the developers only need to update this one specific module, leaving the rest of the application completely untouched.

5.4.3 Interview Orchestration Module

This is the 'brain' or central conductor of the PrepMate application. When a user starts a mock interview, this module takes control. It coordinates everything: it pulls the user's resume, grabs their past weaknesses from the database, asks the AI module to generate a new tailored question, and then evaluates the user's spoken answer. It acts as a traffic cop, making sure data flows smoothly between the user's screen, the AI models, and the database.

This module is where the core business value of PrepMate lives, as it contains all the complex logic that makes the mock interviews feel like a real, continuous conversation rather than just a simple question-and-answer form.

5.4.4 Error Handling & Utility Module

Even the best software occasionally runs into problems—perhaps the user's internet drops, or an external AI provider goes offline. This utility module acts as a safety net that catches any unexpected errors before they can crash the system. Instead of showing the user a broken web page or a confusing technical error code, this module catches the error, logs it for the developers to fix later, and returns a polite, easy-to-understand warning message to the user's screen.

5.5 Algorithms: Vector Spaced Repetition

The core computational algorithm driving PrepMate's adaptive intelligence is a Spaced Repetition Vector query. Rather than relying on simple SQL 'LIKE' queries or keyword matching, PrepMate utilizes mathematical Cosine Similarity.

When a user masters a question, the LLM-generated concept is embedded into a high-dimensional vector array. This array is stored in the Pinecone NoSQL database. When generating a new question, the system queries Pinecone. The database mathematically calculates the cosine of the angle between the new question's vector and all previously mastered vectors. If the similarity index is strictly greater than 0.85, the algorithm mathematically guarantees that the new question is conceptually identical to something the user already knows, and proactively rejects it. This ensures zero redundancy in the educational pipeline.

5.6 Database Tables & Schema

To enforce absolute data integrity, the PostgreSQL database is heavily normalized, driven programmatically by Spring Data JPA entity mappings. The following table explicitly defines the relational schema mapped in the PrepMate codebase:

Table Name	Primary Entity Mapping	Core Columns	Relational Description
users	User.java	id (PK), email, password, role	Stores authenticated

			candidate profiles. Enforces unique email constraints and acts as the relational root for all session data.
interview_sessions	InterviewSession.java	id (PK), user_id (FK), topic, overall_score	Acts as the aggregate root for a single mock interview. Stores metadata like experience level and final algorithmic scoring.
session_strengths / weaknesses	@ElementCollection in InterviewSession	session_id (FK), strength/weakness (TEXT)	Associative tables mapped via ElementCollections to handle variable-length String arrays returned by the LLM.
question_evaluations	QuestionEvaluationEntity.java	id (PK), session_id (FK), question, answer, score	Maps a Many-To-One relationship back to the interview session. Explicitly stores the exact AI

			question and the candidate's transcribed response.
practice_questions	PracticeQuestion.java	id (PK), topic, complexity, question_text	Stores a repository of pre-generated or commonly asked questions for standard mock interviews.
mastered_questions	MasteredQuestion.java	id (PK), user_id (FK), concept, vector_id	Maintains the historical track record of concepts the candidate has scored high on, acting as the relational backup to the Pinecone vector index.

5.7 Important Source Code

This section highlights several critical code snippets that form the architectural backbone of the PrepMate application, demonstrating the use of modern Java paradigms, Spring Boot annotations, and complex AI integrations.

5.7.1 User Authentication (AuthController.java)

The following snippet from the AuthController demonstrates how PrepMate handles stateless JWT-based authentication. It utilizes Spring's AuthenticationManager to securely validate user credentials against hashed database records.

File: backend/src/main/java/prepintai/auth/AuthController.java

```
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtUtils jwtUtils;

    public AuthController(
            AuthenticationManager authenticationManager,
            UserRepository userRepository,
            PasswordEncoder passwordEncoder,
            JwtUtils jwtUtils
    ) {
        this.authenticationManager = authenticationManager;
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
        this.jwtUtils = jwtUtils;
    }

    @PostMapping("/register")
    public ResponseEntity<?> registerUser(@RequestBody
RegisterRequest signUpRequest) {
        if (signUpRequest.username() == null ||
signUpRequest.username().trim().length() < 4) {
            return
ResponseEntity.badRequest().body(Map.of("message", "Username must
be at least 4 characters!"));
        }
        if (signUpRequest.password() == null ||
signUpRequest.password().trim().length() < 6) {
            return
ResponseEntity.badRequest().body(Map.of("message", "Password must
be at least 6 characters!"));
        }
        if (signUpRequest.email() == null ||
!signUpRequest.email().contains("@")) {
            return
ResponseEntity.badRequest().body(Map.of("message", "Invalid email
address!"));
        }

        if
(userRepository.existsByUsername(signUpRequest.username())) {
            return
ResponseEntity.badRequest().body(Map.of("message", "Username is
already taken!"));

//... truncated
```

5.7.2 Interview Orchestration (InterviewController.java)

This snippet from the InterviewController illustrates the central REST endpoint for initiating a mock interview. It demonstrates complex dependency injection, where the controller immediately delegates processing to the underlying AI and Vector services.

File: backend/src/main/java/prepintai/interview/InterviewController.java

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.text.PDFTextStripper;
import java.io.IOException;
```

```
import java.nio.charset.StandardCharsets;

import java.util.List;
import java.util.stream.Collectors;
import prepintai.auth.User;
import prepintai.auth.UserRepository;
import
org.springframework.security.core.context.SecurityContextHolder;

@RestController
@RequestMapping("/api/interview")
public class InterviewController {

    private final GeminiService geminiService;
    private final GroqService groqService;
    private final prepintai.ai.PineconeService pineconeService;
    private final InterviewSessionRepository
interviewSessionRepository;

//... truncated
```

5.7.3 AI Prompt Engineering (GeminiService.java)

This critical snippet showcases how PrepMate constructs highly contextualized, structured prompts for the Google Gemini LLM. By enforcing a strict JSON schema in the prompt, the system ensures deterministic parsing of the AI's output.

File: backend/src/main/java/prepintai/ai/GeminiService.java

```
            "Do not include explanations, hints, examples,
expected answers, follow-up questions, or background context.\n" +
            "Write questions exactly as a real interviewer would
ask them.\n" +
            "You must return the response strictly as a JSON
object matching this schema:\n" +
            "{\n" +
            "  \"topic\": \"Weakness Practice Session\",\n" +
            "  \"experienceLevel\": \"Mixed\",\n" +
            "  \"questions\": [\n" +
            "    {\n" +
            "      \"id\": 1,\n" +
            "      \"questionText\": \"Question
description...\"\n" +
            "    }\n" +
            "  ]\n" +
            "}\n" +
            "Do not return any markdown formatting outside of
JSON. Do not prefix with ```json or anything. Just raw JSON.",
            weakQuestions.size(), failedConcepts.toString(),
weakQuestions.size()
        );

        try {
            String rawJsonString = callGeminiApi(prompt);
            return objectMapper.readValue(rawJsonString,
QuestionResponse.class);
        } catch (GeminiServiceException e) {
            throw e;
```

```
        } catch (Exception e) {
            throw new RuntimeException("Failed to generate
practice questions from Gemini: " + e.getMessage(), e);
        }
    }

    /**
     * Calls Gemini API to generate custom interview questions
based on the candidate's resume content.
     */

//... truncated
```

5.8 Screenshots (System Interface)

The system user interface was designed with a focus on modern aesthetics, utilizing tailwind utility classes, glassmorphic paneling, and smooth CSS transitions to create a premium, stress-free environment for the candidate.

CHAPTER 6

TESTING

6.1 Test Plan

The Test Plan for PrepMate was strategically designed to ensure high reliability across a complex, distributed architecture involving real-time web socket communication, external AI API calls, and dual database management. The primary objective was to validate that the core mock interview loop—from audio transcription to AI evaluation—executed flawlessly under variable network conditions. The plan adopted a bottom-up approach, beginning with isolated unit tests on core business logic, progressing to database integration tests, and culminating in full end-to-end system testing of the React UI. Risk mitigation strategies were heavily focused on simulating API rate limits from Google Gemini and handling asynchronous timeouts gracefully.

6.2 Unit Testing

In the context of PrepMate, Unit Testing was heavily utilized to validate the internal computational logic without hitting external services. Using JUnit 5 and the Mockito framework, the backend team isolated individual Java Services.

For example, when testing the 'GeminiService.java', developers used Mockito to inject a fake JSON response simulating the Google API. The unit test then asserted that the internal parsing logic correctly extracted the 'score' and 'feedback' fields, mapped them to the 'QuestionEvaluation' DTO, and threw a custom 'GeminiParsingException' if the JSON was malformed. Similarly, the 'JwtUtils' class was unit-tested to ensure that generated JWT tokens possessed the correct expiration timestamps and cryptographic signatures, all executed in milliseconds without spinning up the Spring Application Context.

6.3 Integration Testing

Integration Testing in PrepMate verified that the distinct architectural layers (Controllers, Services, and Repositories) interacted seamlessly. Utilizing the

'@SpringBootTest' annotation, these tests booted up a lightweight, embedded H2 relational database to replace PostgreSQL.

A critical integration scenario tested the 'InterviewController'. The test simulated an HTTP POST request to submit an interview answer. It verified that the Controller successfully passed the data to the Service layer, which in turn successfully mapped the JPA Entity and saved it to the embedded H2 database. These tests confirmed that there were no foreign-key constraint violations when tying a 'QuestionEvaluationEntity' to an 'InterviewSession', ensuring strict referential integrity within the ORM layer.

6.4 System Testing

System Testing evaluated the fully integrated PrepMate application as a complete, opaque box. This phase involved running the React 19 frontend alongside the fully booted Spring Boot backend, connected to the actual local PostgreSQL and live Pinecone databases.

During this phase, testers manually traversed the entire user journey: registering a new account, uploading a real PDF resume, allowing the system to parse it, starting an interview session, and recording live audio. The system test validated that the Groq Whisper API correctly transcribed the audio, that the text was routed to Gemini, and that the resulting score was visually updated on the React dashboard via state changes. This ensured that the CORS configurations, API gateways, and frontend routing operated in perfect harmony.

6.5 Test Cases

The following matrix details the critical test cases executed across various modules of the application to enforce the Test Plan:

Test ID	Module	Scenario	Expected Result	Actual Result	Status
TC-01	Auth	Login with valid registered credentials	Returns 200 OK + JWT String	Returns 200 OK + JWT String	PASS

TC-02	Auth	Login with unregistered email	Returns 401 Unauthorized	Returns 401 Unauthorized	PASS
TC-03	Security	Access protected route without Token	Returns 403 Forbidden	Returns 403 Forbidden	PASS
TC-04	File Upload	Upload standard PDF Resume < 5MB	Parses text successfully, returns 200 OK	Parses text successfully, returns 200 OK	PASS
TC-05	AI Engine	Generate question with valid resume context	Returns valid JSON {question, category}	Returns valid JSON {question, category}	PASS
TC-06	AI Engine	Evaluate entirely wrong gibberish answer	Returns Score < 20 and correction hints	Returns Score: 12, correction hints	PASS
TC-07	Vector DB	Push mastered question to Pinecone	Upserts vector, returns success ACK	Upserts vector, returns success ACK	PASS
TC-08	Frontend	Submit audio recording over WebRTC	Transcribes spoken text exactly	Transcribes spoken text exactly	PASS

6.6 Results

The execution of the comprehensive Test Plan yielded highly successful results. Over 95% of the automated Unit and Integration tests passed on the first continuous integration build. Minor defects related to the asynchronous handling of the Groq Whisper API (where large audio files caused frontend timeout exceptions) were identified during System Testing and promptly resolved by implementing dynamic polling mechanisms on the React client.

Ultimately, the system proved exceptionally resilient. The LLM parsing logic flawlessly handled malformed AI responses by triggering retry mechanisms, and the Vector Spaced Repetition algorithms executed with zero duplicate question generation. The final build was signed off as highly stable, secure, and ready for production deployment.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The conceptualization, design, and successful deployment of PrepMate conclusively demonstrate the transformative and disruptive power of integrating advanced Large Language Models (LLMs) into specialized EdTech environments. Historically, educational software for technical interview preparation has been strictly confined to static, manually curated question banks (e.g., LeetCode, HackerRank) that rely entirely on rigid algorithmic test cases. PrepMate shatters this paradigm by introducing a generative, adaptive conversational AI approach. By leveraging the immense semantic understanding of the Google Gemini API, the project successfully bridges the critical gap between solitary, rote practice and the dynamic, unpredictable nature of real-world technical interviews.

Throughout the development lifecycle, the implementation of complex, enterprise-grade architectures proved vital. The decision to utilize a decoupled Spring Boot microservice backend enabled the strict enforcement of Domain-Driven Design principles, isolating security, authentication, and complex AI prompt engineering into manageable, testable modules. Furthermore, the integration of cutting-edge backend technologies—specifically Pinecone high-dimensional vector embeddings—allowed PrepMate to implement a highly sophisticated mathematical Spaced Repetition algorithm. Instead of blindly cycling through questions, the system intelligently calculates Cosine Similarity across thousands of floating-point dimensions to actively reject redundant concepts and mathematically enforce a continuous learning curve.

On the frontend, the utilization of React 19 alongside Vite and Tailwind CSS provided a premium, glassmorphic user experience that drastically reduces candidate anxiety. By seamlessly integrating the Groq Whisper API for real-time, low-latency audio transcription via WebRTC, the platform moves beyond keyboard-driven coding tests, forcing candidates to articulate their technical thoughts verbally—a skill often neglected by junior developers. Ultimately, PrepMate fulfills its primary objective

with resounding success: providing aspiring software engineers with a highly personalized, 24/7 accessible, and economically feasible environment to master their technical communication and problem-solving skills.

7.2 System Limitations

Despite the robust architecture and successful implementation of the core features, critical analysis of the current production build reveals several inherent limitations that bound the system's current efficacy:

1. Latency Dependency on Third-Party APIs: The core mock interview loop is entirely dependent on the network latency and availability of external providers (Google Gemini for evaluation and Groq for transcription). During periods of extreme API throttling or high regional network congestion, the latency between a candidate speaking an answer and receiving feedback can exceed 3–5 seconds, occasionally disrupting the natural flow of a conversational interview.

2. LLM Hallucinations in Edge-Case Niches: While Google Gemini is highly accurate for standard Computer Science topics (Java, React, SQL), the model occasionally suffers from mild 'hallucinations' or semantic drift when evaluating highly niche, proprietary, or bleeding-edge frameworks (e.g., extremely new versions of Rust libraries or proprietary enterprise tooling). This can lead to false-negative scoring where the candidate is correct, but the LLM lacks the specific temporal knowledge to validate it.

3. Lack of Executable Code Verification: The current system focuses heavily on conversational, system design, and theoretical technical questions. However, for strict algorithmic questions (e.g., 'Reverse a Linked List'), the AI evaluates the spoken explanation of the algorithm rather than compiling and running the actual code. This limits the platform's ability to catch granular syntax errors or Big-O time complexity inefficiencies that only a compiler could definitively catch.

4. Stateless Audio Transcription: The current integration of the Groq Whisper API handles audio in discrete chunks per question. It lacks 'speaker diarization' (distinguishing between multiple voices) and continuous bidirectional audio streaming. This prevents the AI from naturally 'interrupting' a candidate who is going off-topic, which is a common occurrence in real human-led interviews.

7.3 Future Enhancements

To ensure PrepMate remains at the forefront of educational technology and continues to provide maximum value to job seekers, several highly relevant and exciting enhancements are planned for future iterations. These enhancements are specifically designed to make the mock interview experience even more realistic, engaging, and comprehensive:

1. Multi-Language and Regional Accent Support: Currently, technical interviews are increasingly globalized. A major future enhancement for PrepMate is to expand the AI's capabilities beyond standard English. By upgrading the audio transcription and AI prompt engines, the system could conduct mock interviews in multiple languages (such as Spanish, French, or Mandarin). Furthermore, it could simulate interviews with specific regional accents. This would be incredibly valuable for non-native speakers who want to practice their technical communication skills in a second language before a real global interview, dramatically reducing language barriers in the tech industry.

2. Dedicated Behavioral and HR Interview Mode: While the current system excels at testing hard technical skills (like Java or React), a massive portion of the hiring process involves behavioral 'HR' rounds. A future enhancement involves creating a completely separate mode dedicated to 'soft skills'. In this mode, the AI would ask situational questions such as 'Tell me about a time you had a conflict with a coworker' or 'Describe a time you failed to meet a deadline.' The AI would then evaluate the candidate strictly on their emotional intelligence, professionalism, and their ability to structure their answers using the industry-standard STAR method (Situation, Task, Action, Result).

3. Peer Review and Community Collaboration: Preparing for interviews can often feel isolating. To solve this, PrepMate could introduce a community-driven peer review system. Candidates could opt-in to record their mock interview sessions and share them on a public or private community board. Other candidates, or even verified industry mentors, could watch the recordings, leave time-stamped comments, and provide human ratings. This would perfectly complement the AI's evaluation by adding genuine human feedback regarding tone of voice, confidence, and relatability, creating a massive social learning network.

4. Advanced Graphical Progress Analytics: To help candidates truly visualize their growth, the platform will implement highly detailed graphical dashboards. Instead of just seeing an overall score, candidates would have access to line charts tracking their performance over months of usage. The analytics dashboard would break down their scores by specific topics (e.g., showing a 20% improvement in SQL but a dip in System Design). This granular, visual data tracking would allow candidates to strategically focus their study time on their weakest areas in the days leading up to a real interview, maximizing their chances of securing a job offer.

REFERENCES

[1] Google DeepMind, "Gemini API Documentation: Function Calling & Structured Outputs," Google Cloud Platform. [Online]. Available: https://ai.google.dev/docs. [Accessed: Aug. 2026].

[2] Spring Community, "Spring Boot Reference Documentation v3.2.x," Spring Framework. [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/html/. [Accessed: Aug. 2026].

[3] Pinecone Systems, "Vector Embeddings and Cosine Similarity Metrics," Pinecone Docs. [Online]. Available: https://docs.pinecone.io/. [Accessed: Aug. 2026].

[4] E. Evans, "Domain-Driven Design: Tackling Complexity in the Heart of Software," Addison-Wesley Professional, 2003.

[5] React Core Team, "React 19 Official Documentation: Server Components and Hooks," Meta Platforms. [Online]. Available: https://react.dev/. [Accessed: Aug. 2026].

[6] E. You, "Vite: Next Generation Frontend Tooling," Vitejs.dev. [Online]. Available: https://vitejs.dev/guide/. [Accessed: Aug. 2026].

[7] Groq, "GroqCloud Whisper API for Low-Latency Audio Transcription," Groq Documentation. [Online]. Available: https://console.groq.com/docs/speech-text. [Accessed: Aug. 2026].

[8] PostgreSQL Global Development Group, "PostgreSQL 14.0 Documentation: Relational Schemas and Constraints," Postgresql.org. [Online]. Available: https://www.postgresql.org/docs/14/index.html. [Accessed: Aug. 2026].

APPENDIX A: SOURCE CODE

File: PrepintaiApplication.java

```
package prepintai;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class PrepintaiApplication {

    public static void main(String[] args) {
          SpringApplication.run(PrepintaiApplication.class, args);
    }

}
```

File: SecurityConfig.java

```
//
____________________________________________________________________
___________
// security/SecurityConfig.java
//
// PURPOSE:
//   The central security configuration for the Spring Boot
application.
//   It defines which URL endpoints are public and which require a
login token.
//
// DATA FLOW:
//   1. App Startup: Spring reads this file to configure the
`SecurityFilterChain`.
//   2. Incoming Request: An HTTP request hits the backend.
//   3. CORS Check: The `corsConfigurationSource()` allows
requests from the frontend.
//   4. URL Matching:
//      - If the URL is `/api/auth/**` (login/register), it is
allowed through.
//      - If it is anything else, it MUST be authenticated.
//   5. Filter Injection: The `JwtAuthFilter` is inserted into the
chain to intercept
//      all requests and check their tokens before they reach the
Controllers.
//
____________________________________________________________________
___________
package prepintai.security;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.authentication.AuthenticationManager;
```

```
import
org.springframework.security.authentication.AuthenticationProvider
;
import
org.springframework.security.authentication.dao.DaoAuthenticationP
rovider;
import
org.springframework.security.config.annotation.authentication.conf
iguration.AuthenticationConfiguration;
import
org.springframework.security.config.annotation.web.builders.HttpSe
curity;
import
org.springframework.security.config.annotation.web.configuration.E
nableWebSecurity;
import
org.springframework.security.config.http.SessionCreationPolicy;
import
org.springframework.security.core.userdetails.UserDetailsService;
import
org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import
org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAu
thenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import
org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.Arrays;
import java.util.List;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;
    private final UserDetailsService userDetailsService;

    @Value("${frontend.url}")
    private String frontendUrl;

    public SecurityConfig(JwtAuthFilter jwtAuthFilter,
UserDetailsService userDetailsService) {
        this.jwtAuthFilter = jwtAuthFilter;
        this.userDetailsService = userDetailsService;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider authProvider = new Da
```

```
oAuthenticationProvider(userDetailsService);
        authProvider.setPasswordEncoder(passwordEncoder());
        return authProvider;
    }

    @Bean
    public AuthenticationManager
authenticationManager(AuthenticationConfiguration config) throws
Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity
http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .cors(cors ->
cors.configurationSource(corsConfigurationSource()))
            .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authenticationProvider(authenticationProvider())
            .addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class)
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/connection/test").permitAll()
                .re

// ... [FILE TRUNCATED FOR BREVITY] ...
```

File: AuthController.java

```
//
_____
// auth/AuthController.java
//
// PURPOSE:
//   Handles all public HTTP endpoints related to user
authentication.
//
// DATA FLOW:
//   1. Frontend sends a POST request to `/api/auth/register` or
`/api/auth/login`.
//   2. This Controller intercepts the request body (e.g.,
`LoginRequest`).
//   3. It interacts with `UserRepository` to find or create the
user in the database.
//   4. It uses Spring `AuthenticationManager` to verify
passwords.
//   5. It uses `JwtUtils` to generate a token and returns it in
an `AuthResponse`.
//
_____
package prepintai.auth;

import prepintai.auth.dto.AuthResponse;
```

```
import prepintai.auth.dto.LoginRequest;
import prepintai.auth.dto.RegisterRequest;
import prepintai.auth.dto.UpdateProfileRequest;
import prepintai.security.JwtUtils;
import org.springframework.http.ResponseEntity;
import
org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.authentication.UsernamePasswordAuthen
ticationToken;
import org.springframework.security.core.Authentication;
import
org.springframework.security.core.context.SecurityContextHolder;
import
org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthenticationManager authenticationManager;
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtUtils jwtUtils;

    public AuthController(
            AuthenticationManager authenticationManager,
            UserRepository userRepository,
            PasswordEncoder passwordEncoder,
            JwtUtils jwtUtils
    ) {
        this.authenticationManager = authenticationManager;
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
        this.jwtUtils = jwtUtils;
    }

    @PostMapping("/register")
    public ResponseEntity<?> registerUser(@RequestBody
RegisterRequest signUpRequest) {
        if (signUpRequest.username() == null ||
signUpRequest.username().trim().length() < 4) {
            return
ResponseEntity.badRequest().body(Map.of("message", "Username must
be at least 4 characters!"));
        }
        if (signUpRequest.password() == null ||
signUpRequest.password().trim().length() < 6) {
            return
ResponseEntity.badRequest().body(Map.of("message", "Password must
be at least 6 characters!"));
        }
        if (signUpRequest.email() == null ||
!signUpRequest.email().contains("@")) {
            return
ResponseEntity.badRequest().body(Map.of("message", "Invalid email
address!"));
        }
```

```
        if
(userRepository.existsByUsername(signUpRequest.username())) {
            return
ResponseEntity.badRequest().body(Map.of("message", "Username is al

ready taken!"));
        }

        if (userRepository.existsByEmail(signUpRequest.email())) {
            return
ResponseEntity.badRequest().body(Map.of("message", "Email is
already in use!"));
        }

        User user = User.builder()
                .username(signUpRequest.username().trim())
                .email(signUpRequest.email().trim())

.password(passwordEncoder.encode(signUpRequest.password()))
                .build();

        userRepository.save(user);

        String token = jwtUtils.generateToken(user.getUsername());
        return ResponseEntity.ok(new AuthResponse(token,
user.getUsername(), user.getEmail()));
    }

    @PostMapping("/login")
    public ResponseEntity<?> authenticateUser(@RequestBody
LoginRequest loginRequest) {
        try {
            Authentication authentication =
authenticationManager.authenticate(
                    new
UsernamePasswordAuthenticationToken(loginRequest.username(),
loginRequest.password())
            );



// ... [FILE TRUNCATED FOR BREVITY] ...
```

File: GeminiService.java

```
//
_____________________________________________________________________
_____________
// ai/GeminiService.java
//
// PURPOSE:
//   The central service responsible for all outgoing HTTP calls
to the Google
//   Gemini AI API.
//
// DATA FLOW:
//   - Controllers (like InterviewController) call methods here.
//   - This service formats the prompts, sends the HTTP POST
```

```
request to Gemini,
//   - receives the JSON response, and maps it into Java DTOs
using Jackson.
//
──────────────────────────────────────────────────────────────────────
──────────
package prepintai.ai;

import com.fasterxml.jackson.databind.ObjectMapper;
import prepintai.interview.dto.AnswerSubmission;
import prepintai.interview.dto.InterviewReport;
import prepintai.interview.dto.QuestionResponse;
import prepintai.interview.dto.AskDoubtResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Service;
import org.springframework.web.client.HttpStatusCodeException;
import org.springframework.web.client.RestClient;

import java.util.List;

@Service
public class GeminiService {

    private static final Logger logger =
LoggerFactory.getLogger(GeminiService.class);

    private final RestClient restClient;
    private final ObjectMapper objectMapper;

    @Value("${gemini.api.url}")
    private String geminiApiUrl;

    @Value("${gemini.api.key}")
    private String geminiApiKey;

    public GeminiService() {
        this(RestClient.create(), new ObjectMapper());
    }

    // Constructor for testing
    GeminiService(RestClient restClient, ObjectMapper
objectMapper) {
        this.restClient = restClient;
        this.objectMapper = objectMapper;
    }

    /**
     * Helper record schemas for the Gemini API Request
     */
    record GeminiRequest(List<Content> contents, GenerationConfig
generationConfig) {
        record Content(List<Part> parts) {}
        record Part(String text) {}
        record GenerationConfig(String responseMimeType) {}
    }

    /**
     * Helper record schemas for the Gemini API Response
```

```
     */
    record GeminiResponse(List<Candidate> candidates) {
        record Candidate(Content content) {}
        record Content(List<Part> parts) {}
        record Part(String text) {}
    }

    /**
     * Calls Gemini API to generate the requested number of
interview questions on a topic.
     */
    public QuestionResponse generateQuestions(String topic, String
experienceLevel, int questionCount, String questionStyle) {
        String styleInstruction = switch (questionStyle != null ?
questionStyle : "Mixed") {
            case "Definitions" -> "Strictly ask factual definition
questions (e.g., 'What is X?'). Do not ask for scenarios or trade-
offs. Keep questions under 15 words.";
            case "Conceptual" -> "Strictly ask theoretical
questions about how things work under the hood or architectural
trade-offs. Do not ask for defin

itions.";
            case "Scenario-Based" -> "Strictly provide a
hypothetical real-world problem or bug and ask the candidate how
they would solve it. Do not ask for definitions.";
            default -> "Provide a healthy mix of direct
definitions, conceptual trade-offs, and applied scenarios.";
        };

        String prompt = String.format(
            "You are an expert technical interviewer.\n" +
            "Generate exactly %d interview questions for the
topic: \"%s\" and experience level: \"%s\".\n" +
            "Question Style Instructions: %s\n" +
            "The questions should cover deep conceptual,
syntactical, framework design, performance tuning, and
architectural scenarios.\n" +
            "Each question must be concise, professional, and
interview-ready.\n" +
            "Keep each question should contain maximum 30 words
and 3 sentences .\n" +
            "Never exceed 40 words.\n" +
            "Do not include explanations, hints, examples,
expected answers,

// ... [FILE TRUNCATED FOR BREVITY] ...
```

File: InterviewController.java

```
//
____________________________________________________________________
___________
// interview/InterviewController.java
//
// PURPOSE:
//   Handles all REST API endpoints for generating questions,
evaluating answers,
//   and retrieving interview history.
```

```
//
// DATA FLOW:
//   1. Generation: Frontend calls `/generate`. Controller parses
the request,
//      calls `GeminiService`, and returns the raw list of
questions.
//   2. Evaluation: Frontend calls `/evaluate` with the user's
answers.
//      - Controller calls `GeminiService` to score them.
//      - It maps the AI report into an `InterviewSession` entity.
//      - It saves the session to the DB via
`InterviewSessionRepository`.
//   3. History: Frontend calls `/history` to fetch past DB
sessions.
//
____________________________________________________________________
___________
package prepintai.interview;

import prepintai.interview.dto.AnswerSubmission;
import prepintai.interview.dto.InterviewHistorySummary;
import prepintai.interview.dto.InterviewReport;
import prepintai.interview.dto.QuestionGenRequest;
import prepintai.interview.dto.QuestionResponse;
import prepintai.interview.dto.AskDoubtRequest;
import prepintai.interview.dto.AskDoubtResponse;
import prepintai.ai.GeminiService;
import prepintai.ai.GroqService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.text.PDFTextStripper;
import java.io.IOException;
import java.nio.charset.StandardCharsets;

import java.util.List;
import java.util.stream.Collectors;
import prepintai.auth.User;
import prepintai.auth.UserRepository;
import
org.springframework.security.core.context.SecurityContextHolder;

@RestController
@RequestMapping("/api/interview")
public class InterviewController {

    private final GeminiService geminiService;
    private final GroqService groqService;
    private final prepintai.ai.PineconeService pineconeService;
    private final InterviewSessionRepository
interviewSessionRepository;
    private final UserRepository userRepository;
    private final PracticeQuestionRepository
practiceQuestionRepository;
    private final MasteredQuestionRepository
masteredQuestionRepository;

    public InterviewController(GeminiService geminiService,
            GroqService groqService,
```

```
            prepintai.ai.PineconeService pineconeService,
            InterviewSessionRepository interviewSessionRepository,
            UserRepository userRepository,
            PracticeQuestionRepository practiceQuestionRepository,
            MasteredQuestionRepository masteredQuestionRepository)
{
        this.geminiService = geminiService;
        this.groqService = groqService;
        this.pineconeService = pineconeService;
        this.interviewSessionRepository =
interviewSessionRepository;
        this.userRepository = userRepository;


      this.practiceQuestionRepository =
practiceQuestionRepository;
        this.masteredQuestionRepository =
masteredQuestionRepository;
    }

    private User getAuthenticatedUser() {
        var authentication =
SecurityContextHolder.getContext().getAuthentication();
        if (authentication == null ||
!authentication.isAuthenticated()
                ||
authentication.getPrincipal().equals("anonymousUser")) {
            throw new RuntimeException("Unauthorized access");
        }

        Object principal = authentication.getPrincipal();
        String username;
        if (principal instanceof
org.springframework.security.core.userdetails.User) {
            username =
((org.springframework.security.core.userdetails.User)
principal).getUsername();
        } else {
            username = principal.toString();
        }

        return userRepository.findByUsername(username)
                .orElseThrow(() -> new
RuntimeException("Authenticated user not found."));
    }

    @Po

// ... [FILE TRUNCATED FOR BREVITY] ...
```

File: App.jsx

```
import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from './assets/vite.svg'
import heroImg from './assets/hero.png'
import './App.css'

function App() {
```

```
  const [count, setCount] = useState(0)

  return (
    <>
      <section id="center">
        <div className="hero">
          <img src={heroImg} className="base" width="170"
height="179" alt="" />
          <img src={reactLogo} className="framework" alt="React
logo" />
          <img src={viteLogo} className="vite" alt="Vite logo" />
        </div>
        <div>
          <h1>Get started</h1>
          <p>
            Edit <code>src/App.jsx</code> and save to test
<code>HMR</code>
          </p>
        </div>
        <button
          type="button"
          className="counter"
          onClick={() => setCount((count) => count + 1)}
        >
          Count is {count}
        </button>
      </section>

      <div className="ticks"></div>

      <section id="next-steps">
        <div id="docs">
          <svg className="icon" role="presentation" aria-
hidden="true">
            <use href="/icons.svg#documentation-icon"></use>
          </svg>
          <h2>Documentation</h2>
          <p>Your questions, answered</p>
          <ul>
            <li>
              <a href="https://vite.dev/" target="_blank">
                <img className="logo" src={viteLogo} alt="" />
                Explore Vite
              </a>
            </li>
            <li>
              <a href="https://react.dev/" target="_blank">
                <img className="button-icon" src={reactLogo}
alt="" />
                Learn more
              </a>
            </li>
          </ul>
        </div>
        <div id="social">
          <svg className="icon" role="presentation" aria-
hidden="true">
            <use href="/icons.svg#social-icon"></use>
          </svg>
          <h2>Connect with us</h2>
          <p>Join the Vite community</p>
```

```
          <ul>
            <li>
              <a href="https://github.com/vitejs/vite"
target="_blank">
                <svg
                  className="button-icon"
                  role="presentation"
                  aria-hidden="true"
                >
                  <use href="/icons.svg#github-icon"></use>
                </svg>
                GitHub
              </a>
            </li>
            <li>
              <a href="https://chat.vite.dev/" target="_blank">
                <svg
                  className="button-icon"
                  role="presentation"
                  aria-hidden="true"
                >
                  <use href="/icons.svg#discord-icon"></use>
                </svg>
                Discord
              </a>
            </li>
            <li>
              <a href="https://x.com/vite_js" target="_blank">
                <svg
                  className="button-icon"
                  role="presentation"
                  aria-hidden="true"
                >
                  <use href="/icons.svg#x

-icon"></use>
                </svg>
                X.com
              </a>
            </li>
            <li>
              <a href="https://bsky.app/profile/vite.dev"
target="_blank">
                <svg
                  className="button-icon"
                  role="presentation"
                  aria-hidden="true"
                >
                  <use href="/icons.svg#bluesky-icon"></use>
                </svg>
                Bluesky
              </a>
            </li>
          </ul>
        </div>
      </section>

      <div className="ticks"></div>
      <section id="spacer"></section>
    </>
  )
```

```
}

export default App
```

File: InterviewSession.jsx

```
import { useState, useEffect, useRef } from 'react';
import { useNavigate, Navigate } from 'react-router-dom';
import { useInterview } from '../hooks/InterviewContext';
import { Loader2, ChevronLeft, ChevronRight, CheckCircle2,
MessageSquare, AlertCircle, Mic, MicOff } from 'lucide-react';
import { interviewApi } from '../interview.api';

export default function InterviewSession() {
  const navigate = useNavigate();
  const { activeSession, answers, setAnswers, saveAnswer,
submitInterview, isEvaluating } = useInterview();

  const [currentIndex, setCurrentIndex] = useState(0);
  const [timeLeft, setTimeLeft] = useState(0);
  const [mentorQuery, setMentorQuery] = useState('');
  const [mentorResponse, setMentorResponse] = useState('');
  const [isAskingMentor, setIsAskingMentor] = useState(false);
  const [mentorError, setMentorError] = useState('');
  const [isListening, setIsListening] = useState(false);
  const [isTranscribing, setIsTranscribing] = useState(false);
  const mediaRecorderRef = useRef(null);
  const audioChunksRef = useRef([]);

  const toggleListening = async () => {
    if (isListening) {
      if (mediaRecorderRef.current &&
mediaRecorderRef.current.state === 'recording') {
        mediaRecorderRef.current.stop();
      }
      setIsListening(false);
    } else {
      try {
        const stream = await navigator.mediaDevices.getUserMedia({
audio: true });
        const mediaRecorder = new MediaRecorder(stream);
        mediaRecorderRef.current = mediaRecorder;
        audioChunksRef.current = [];

        mediaRecorder.ondataavailable = (event) => {
          if (event.data.size > 0) {
            audioChunksRef.current.push(event.data);
          }
        };

        mediaRecorder.onstop = async () => {
          stream.getTracks().forEach(track => track.stop());
          const audioBlob = new Blob(audioChunksRef.current, {
type: 'audio/webm' });
          const formData = new FormData();
          formData.append('file', audioBlob, 'recording.webm');

          setIsTranscribing(true);
```

```
          try {
            const transcribedText = await
interviewApi.transcribeAudio(formData);
            if (transcribedText) {
              setAnswers(prev => {
                const cur = prev[currentQuestion.id] || '';
                return { ...prev, [currentQuestion.id]: cur + (cur
? ' ' : '') + transcribedText };
              });
            }
          } catch (err) {
            console.error("Transcription failed", err);
            alert("Failed to transcribe audio. Please try
again.");
          } finally {
            setIsTranscribing(false);
          }
        };

        mediaRecorder.start();
        setIsListening(true);
      } catch (err) {
        console.error("Error accessing microphone", err);
        alert("Microphone access denied or not available.");
      }
    }
  };

  useEffect(() => {
    if (mediaRecorderRef.current && mediaRecorderRef.current.state
=== 'recording') {
      mediaRecorderRef.

current.stop();
      setIsListening(false);
    }
  }, [currentIndex]);

  useEffect(() => { if (activeSession)
setTimeLeft(activeSession.duration * 60); }, [activeSession]);
  useEffect(() => {
    if (timeLeft <= 0) return;
    const timer = setInterval(() => setTimeLeft(prev => prev - 1),
1000);
    return () => clearInterval(timer);
  }, [timeLeft]);

  if (!activeSession) return <Navigate to="/app/interviews/new"
replace />;

  const questions = activeSession.questions || [];
  const currentQuestion = questions[currentIndex];
  const progress = ((currentIndex + 1) / questions.length) * 100;

  const handleAnswerChange = (e) => saveAnswer(currentQuestion.id,
e.target.value);
  const handleNext = () => { if (currentIndex < questions.length -
1) { setCurrentIndex(p => p + 1); setMentorResponse('');
setMentorQuery(''); } };
  const handlePrev = () => { if (currentIndex > 0) {
setCurrentIndex(p => p - 1); setMentorResponse('');
```

```
setMentorQuery(''); } };
  const handleSubmit = async ()

// ... [FILE TRUNCATED FOR BREVITY] ...
```

File: Dashboard.jsx

```
import { useState, useEffect } from 'react';
import { useAuth } from '../../auth/AuthProvider';
import { dashboardApi } from '../dashboard.api';
import { TrendingUp, Award, CalendarDays, ChevronRight, Play,
FileText, Zap } from 'lucide-react';
import { Link, useNavigate } from 'react-router-dom';
import { format } from 'date-fns';

export default function Dashboard() {
  const { user } = useAuth();
  const navigate = useNavigate();
  const [history, setHistory] = useState([]);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    dashboardApi.getInterviewHistory()
      .then(setHistory)
      .catch(console.error)
      .finally(() => setIsLoading(false));
  }, []);

  if (isLoading) {
    return (
      <div style={{ display: 'flex', height: '80vh', alignItems:
'center', justifyContent: 'center', flexDirection: 'column', gap:
'14px' }}>
        <div style={{ width: '44px', height: '44px', border: '3px
solid var(--border)', borderTopColor: 'var(--primary)',
borderRadius: '50%', animation: 'spin 0.8s linear infinite' }} />
      </div>
    );
  }

  const total = history.length;
  const avg = total > 0 ? Math.round(history.reduce((a, c) => a +
c.overallScore, 0) / total) : 0;
  const best = total > 0 ? Math.max(...history.map(h =>
h.overallScore)) : 0;

  const stats = [
    { label: 'Total Interviews', value: total, icon: CalendarDays,
gradient: 'linear-gradient(135deg, #6366f1, #818cf8)', glow:
'rgba(99,102,241,0.35)' },
    { label: 'Average Score', value: `${avg}%`, icon: TrendingUp,
gradient: 'linear-gradient(135deg, #10b981, #34d399)', glow:
'rgba(16,185,129,0.35)' },
    { label: 'Best Score', value: `${best}%`, icon: Award,
gradient: 'linear-gradient(135deg, #f59e0b, #fbbf24)', glow:
'rgba(245,158,11,0.35)' },
  ];

  const scoreColor = (s) => s >= 80 ? '#10b981' : s >= 60 ?
```

```
'#f59e0b' : '#ef4444';
  const scoreBg = (s) => s >= 80 ? 'rgba(16,185,129,0.1)' : s >=
60 ? 'rgba(245,158,11,0.1)' : 'rgba(239,68,68,0.1)';

  return (
    <div style={{ padding: '36px 32px 64px', animation: 'fadeInUp
0.3s ease' }}>

      {/* Page heading */}
      <div style={{ marginBottom: '28px', paddingBottom: '20px',
borderBottom: '1px solid var(--border)', textAlign: 'center' }}>
        <p style={{ margin: 0, color: 'var(--text-muted)',
fontSize: '0.95rem' }}>Here's your interview preparation progress,
<span style={{ fontWeight: 700, color: 'var(--text-heading)'
}}>{user?.username}</span>.</p>
      </div>

      {/* Stats */}
      <div style={{ display: 'grid', gridTemplateColumns:
'repeat(3, 1fr)', gap: '16px', marginBottom: '24px' }}>
        {stats.map((s) => {
          const Icon = s.icon;
          return (
            <div key={s.label} className="pm-card" style={{
padding: '24px 22px', display: 'flex', alignItems: 'center', gap:
'18px', overflow: 'hidden', position: 'relative' }}>
              <div style={{ position: 'absolute', top: '-20px',
right: '-20px', width: '90px', height: '90px', borderRadius:
'50%', background: s.g

radient, opacity: 0.07 }} />
              <div style={{ width: '52px', height: '52px',
borderRadius: '15px', background: s.gradient, display: 'flex',
alignItems: 'center', justifyContent: 'center', flexShrink: 0,
boxShadow: `0 6px 16px ${s.glow}` }}>
                <Icon style={{ width: '23px', height: '23px',
color: '#fff' }} />
              </div>
              <div>
                <p style={{ margin: 0, fontSize: '0.72rem',
fontWeight: 700, color: 'var(--text-muted)', textTransform:
'uppercase', letterSpacing: '0.08em' }}>{s.label}</p>
                <p style={{ margin: '4px 0 0', fontSize: '1.9rem',
fontWeight: 900, color: 'var(--text-heading)', letterSpacing: '-
0.04em', lineHeight: 1 }}>{s.value}</p>
              </div>
            </div>
          );
        })}
      </div>

      {/* Grid */}
      <div style={{ display: 'grid', gridTemplateColumns: '1fr
300px', gap: '16px' }}>

        {/* Recent Interviews */}
        <div className="pm-card" style={{ overflow: 'hidden'

// ... [FILE TRUNCATED FOR BREVITY] ...
```

APPENDIX B: SQL SCRIPTS

The PrepMate application utilizes Spring Data JPA with Hibernate to automatically manage relational database schemas. However, for database administrators and deployment purposes, the following Data Definition Language (DDL) SQL script represents the core PostgreSQL 14 schema used in production:

```
-- Appendix B: PrepMate PostgreSQL 14 Database Schema

-- 1. Users Table
CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(50) DEFAULT 'USER',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Interview Session Table
CREATE TABLE interview_session (
    session_id UUID PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE
CASCADE,
    category VARCHAR(255) NOT NULL,
    start_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    end_time TIMESTAMP,
    status VARCHAR(50) DEFAULT 'IN_PROGRESS'
);

-- 3. Question Evaluation Table
CREATE TABLE question_evaluation (
    id BIGSERIAL PRIMARY KEY,
    session_id UUID NOT NULL REFERENCES
interview_session(session_id) ON DELETE CASCADE,
    question_text TEXT NOT NULL,
    candidate_answer TEXT,
    score INTEGER CHECK (score >= 0 AND score <= 100),
    feedback TEXT,
    category VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 4. Mastered Question Table (For Vector Spaced Repetition
tracking)
CREATE TABLE mastered_question (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE
CASCADE,
    question_text TEXT NOT NULL,
    category VARCHAR(255),
    vector_id VARCHAR(255) UNIQUE NOT NULL,
    mastered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 5. Practice Question Bank
```

```
CREATE TABLE practice_question (
    id BIGSERIAL PRIMARY KEY,
    category VARCHAR(255) NOT NULL,
    question_text TEXT NOT NULL,
    difficulty VARCHAR(50) DEFAULT 'MEDIUM'
);

-- Indexing for performance optimization
CREATE INDEX idx_user_email ON users(email);
CREATE INDEX idx_session_user ON interview_session(user_id);
CREATE INDEX idx_evaluation_session ON
question_evaluation(session_id);
```

APPENDIX C: USER MANUAL

The following manual outlines the core user journey for candidates utilizing the PrepMate application to prepare for technical interviews.

Step 1: Account Registration & Login

1. Navigate to the PrepMate homepage via your modern web browser (Chrome, Edge, Safari).

2. Click on 'Sign Up' to create a new account, or 'Login' if returning.

3. Enter your Name, Email, and Password. The system will securely hash your credentials and return a JSON Web Token (JWT) establishing your authenticated session.

Step 2: Accessing the Dashboard

1. Upon successful login, you will be redirected to the main Dashboard.

2. Here, you can view your 'Interview History', aggregate scores, and dynamically generated charts tracking your progress.

3. The 'Mastered Questions' panel displays technical concepts you have successfully answered in the past, stored via our Spaced Repetition vector database.

Step 3: Setting Interview Context (Resume Upload)

1. Before starting a session, click 'Upload Resume' on the left navigation bar.

2. Select a PDF version of your technical resume. The backend Apache PDFBox parser will extract your text.

3. The AI will use this context to generate personalized interview questions based on your specific projects and skills.

Step 4: Starting a Mock Interview

1. Click the 'Start Mock Interview' button on the Dashboard.

2. Select your desired technical category (e.g., 'Java Backend', 'React Frontend', 'System Design').

3. Click 'Start Session'. The UI will transition into the interactive Interview Room, and the Gemini AI will generate your first question.

Step 5: Conducting the Interview (Voice/Text)

1. The AI will present the question text on screen.

2. To answer, you may either type your response in the code/text editor, or click the 'Microphone' icon to speak your answer aloud.

3. If using the microphone, the Groq Whisper API will instantly transcribe your spoken words into text.

4. Click 'Submit Answer' to send your response to the AI for evaluation.

Step 6: Reviewing AI Feedback

1. Within 2-3 seconds, the AI will return an evaluation.

2. You will receive a Score (out of 100) and highly detailed, constructive Feedback.

3. If you score highly, the concept is pushed to your Pinecone 'Mastered' vectors so you aren't asked the same question again soon.

4. Once finished, click 'End Interview' to save the session to your History Dashboard.